

UNIVERSIDADE FEDERAL DE GOIÁS & UNIVERSIDADE CATÓLICA DE GOIÁS
ESCOLA DE ENGENHARIA ELÉTRICA E DE COMPUTAÇÃO



INTRODUÇÃO AO PROGRAMA "MATLAB" COM APLICAÇÕES

Material elaborado por:

Prof. Dr. Antônio César Baleeiro Alves (UCG/UFG)

Prof. Dr. Enes Gonçalves Marra (UFG)

Prof. Dr. José Wilson Lima Nerys (UFG)

Goiânia – GO

2007

SUMÁRIO

<u>1</u>	<u>INTRODUÇÃO</u>	<u>5</u>
<u>2</u>	<u>UTILIZANDO O MATLAB</u>	<u>9</u>
2.1	INICIAR O PROGRAMA	9
2.2	PASTA DE TRABALHO	10
2.3	TELA DA LINHA DE COMANDO DO MATLAB	11
2.4	CRIAÇÃO DE ARQUIVOS NO MATLAB	11
2.5	LIMITAÇÕES DO MATLAB	13
<u>3</u>	<u>OPERADORES, CONSTANTES E CARACTERES ESPECIAIS</u>	<u>14</u>
3.1	OPERADORES ARITMÉTICOS	14
3.2	CARACTERES ESPECIAIS	16
3.3	OPERADORES LÓGICOS	18
3.4	OPERADORES RELACIONAIS	19
3.5	CONSTANTES OU VARIÁVEIS	19
3.5.1	INF	20
3.5.2	NAN	20
3.5.3	IEEE	20
3.6	RECURSOS DE APOIO DA TELA DE COMANDO DO MATLAB	20
3.6.1	MENU FILE	20
3.6.2	MENU EDIT	21
<u>4</u>	<u>COMANDOS DE PROPÓSITO GERAL</u>	<u>22</u>
4.1	COMANDOS DE INFORMAÇÃO	22
4.2	COMANDOS DE OPERAÇÃO COM A MEMÓRIA DA ÁREA DE TRABALHO (WORKSPACE)	23
4.3	COMANDOS DE OPERAÇÃO COM FUNÇÕES E COMANDOS	24
4.4	COMANDOS DE OPERAÇÃO COM O CAMINHO DE DIRETÓRIOS	25
4.5	COMANDOS DE OPERAÇÃO COM A JANELA DE COMANDO	26
4.6	COMANDOS DE OPERAÇÃO COM O SISTEMA OPERACIONAL	27
<u>5</u>	<u>COMANDOS BÁSICOS PARA MANIPULAÇÃO DE MATRIZES</u>	<u>29</u>
<u>6</u>	<u>FUNÇÕES MATEMÁTICAS ELEMENTARES</u>	<u>32</u>

	3
<u>7 COMANDOS DE CONTROLE GRÁFICO</u>	<u>36</u>
7.1 GRÁFICOS BIDIMENSIONAIS	36
7.2 GRÁFICOS TRIDIMENSIONAIS	38
<u>8 OUTROS COMANDOS</u>	<u>39</u>
8.1 FUNÇÕES LÓGICAS	39
8.2 FUNÇÕES DE CONTROLE DE FLUXO DE PROGRAMAÇÃO	40
8.3 FUNÇÕES DE CONTROLE DE BAIXO NÍVEL	44
8.4 FUNÇÕES ESPECIALIZADAS: MATEMÁTICAS, MATRICIAIS, ÁLGEBRA LINEAR	45
8.5 FUNÇÕES DE INTEGRAÇÃO, INTERPOLAÇÃO E TRATAMENTO DE DADOS	45
8.6 FUNÇÕES DE CONTROLE DE CORES, IMAGENS E CARACTERES	45
<u>9 PROGRAMANDO EM ARQUIVOS .M</u>	<u>46</u>
9.1 EXEMPLOS DE APLICAÇÃO DE PROGRAMA '.M'	46
<u>10 MATEMÁTICA SIMBÓLICA</u>	<u>51</u>
10.1 USO DA MATEMÁTICA SIMBÓLICA	56
10.2 GRÁFICOS EM DUAS DIMENSÕES	57
10.3 GRÁFICOS EM TRÊS DIMENSÕES	58
<u>11 SIMULINK</u>	<u>60</u>
<u>12 EXERCÍCIOS RESOLVIDOS</u>	<u>69</u>
<u>13 EXERCÍCIOS PROPOSTOS</u>	<u>78</u>
<u>14 BIBLIOGRAFIA</u>	<u>81</u>

1 INTRODUÇÃO

O MATLAB (abreviatura de *Matrix Laboratory*) é um programa para desenvolvimento e implementação de algoritmos numéricos ou simbólicos que oferece ao usuário um ambiente interativo de programação para estudo e pesquisa nas diversas áreas das ciências exatas.

Esse sistema comporta os recursos de linguagem procedural de programação assemelhados aos do FORTRAN, ANSI C e do Pascal, além de capacidade gráfica e a possibilidade de operar com instruções simbólicas. Um dos aspectos mais poderosos é o fato da linguagem MATLAB permitir construir suas próprias ferramentas reutilizáveis. Isto é feito através da escrita de suas próprias funções e programas especiais conhecidos como arquivos `.m`.

A primeira versão do MATLAB foi escrita nas Universidades do Novo México e Stanford, na década de 1970, e destinava-se a cursos de teoria matricial, álgebra linear e análise numérica. Os pacotes para manipulação de sub-rotinas em FORTRAN, denominados LINPACK e EISPACK, foram os precursores do MATLAB.

O MATLAB tem evoluído continuamente, com a contribuição e sugestões de inúmeros usuários. No meio universitário, o MATLAB tornou-se quase que uma ferramenta padrão em cursos introdutórios e avançados de Álgebra Aplicada, Processamento de Sinais, Sistemas de Controle, Estatística e inúmeras outras áreas do conhecimento.

O MATLAB contempla ainda uma grande família de aplicações específicas, as quais são denominadas *Toolboxes* (caixas de ferramentas), como ilustrado na Fig. 1.1. Estes *Toolboxes* são conjuntos abrangentes de funções MATLAB cujo objetivo é resolver problemas de áreas específicas, tais como: Processamento de Sinais, Projeto de Sistemas de Controle, Simulação Dinâmica de Sistemas, Identificação de Sistemas, Redes Neurais, Lógica Fuzzy (nebulosa ou difusa), Otimização de Sistemas, *Wavelets*, Cálculo Simbólico, e outras áreas. Os usos típicos incluem:

- Cálculos matemáticos;
- Desenvolvimento de algoritmos;
- Modelagem, simulação e confecção de protótipos;
- Análise, exploração e visualização de dados;
- Gráficos científicos e de engenharia;

- Desenvolvimento de aplicações, incluindo a elaboração de interfaces gráficas com o usuário

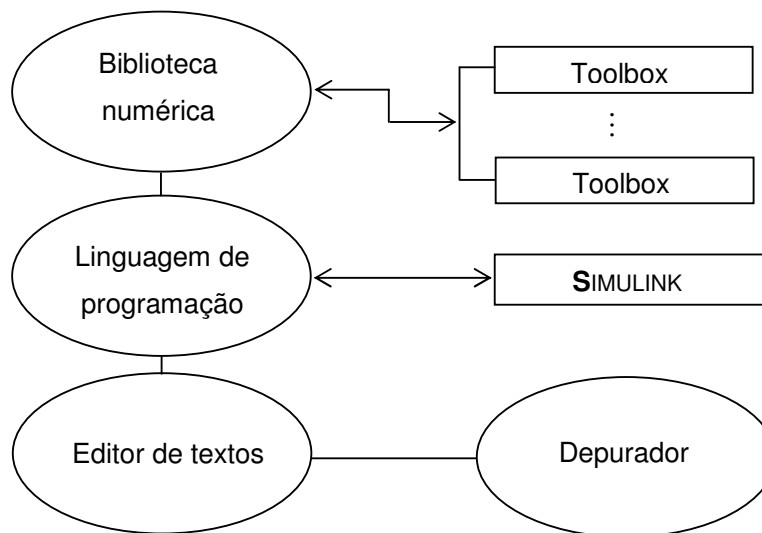


Fig. 1.1: Diagrama de Blocos do MATLAB

A Fig. 1.1 ilustra alguns componentes do sistema MATLAB. Cabe ressaltar, entretanto, que vários Toolboxes estão disponíveis e a cada nova versão do MATLAB novos Toolboxes são adicionados, incluindo as contribuições de usuários espalhados pelo mundo. O **SIMULINK** é um acessório que possui bibliotecas adicionais de blocos para aplicações especiais como comunicações e simulações de circuitos elétricos e eletrônicos.

O MATLAB é disponibilizado para praticamente todos os sistemas operacionais atualmente em uso: Macintosh, Windows 95 e NT, Linux, UNIX da Sun, IBM e Open VMS.

A empresa MathWorks é quem negocia/distribui o MATLAB. Seus endereços são:

Internet: <http://www.mathworks.com>
Telefone (ligação para os EUA): 00-1-508-647-7000

Para um contato com vistas ao uso do programa na área educacional, tente o endereço:

The MathWorks, Inc.
University Sales Department
24 Prime Park Way
Natick, Massachusetts 01760-1500
Telefone: 00-1-508-647-7000
E-mail: info@mathworks.com

A necessidade de utilizar um sistema de computação numérica, e a familiarização com os comandos e as funções do MATLAB vão gradualmente aumentando o potencial de quem programa em MATLAB. No entanto, a bibliografia existente pode ser de grande auxílio. Parte desta bibliografia é editada pela The MathWorks, como por exemplo:

- MATLAB User's Guide (Guia do usuário do MATLAB);
- MATLAB Reference Guide (Guia de referência do MATLAB);
- External Interface Guide (Guia de comunicação externa);
- Installation Guide (Guia de Instalação);
- SIMULINK User's Guide (Guia do usuário do SIMULINK);
- Online Help (Sistema de auxílio do programa MATLAB).

Inúmeras outras referências podem ser encontradas para o MATLAB, para isto basta procurar nas diversas editoras disponíveis no mercado. Algumas destas referências (em português) para a área de controle podem ser:

- Katsuhiko Ogata, Solução de Problemas de Engenharia de Controle com MATLAB. Prentice-Hall do Brasil, Rio de Janeiro, 1997, 330p.
- Katsuhiko Ogata, Projeto de Sistemas Lineares com MATLAB. Prentice-Hall do Brasil, Rio de Janeiro, 1996, 202p.

Na *internet* existem diversas listas de discussão a respeito do MATLAB. Inscrever-se em uma destas lista pode ser uma boa fonte de informação a respeito do MATLAB. Na página da The MathWorks Inc. é possível obter informação sobre

algumas destas listas, bem como sobre o próprio MATLAB. O endereço desta página é: <http://www.mathworks.com>.

Os endereços eletrônicos a seguir podem ser também utilizados para saber mais sobre o MATLAB:

- Perguntas técnicas: tech@mathworks.com
- Sugestões de melhoria: suggest@mathworks.com
- Bug (erros) do MATLAB: bugs@mathworks.com
- Erros nos manuais: doc@mathworks.com
- Registro de produto (após a compra): register@mathoworks.com
- Renovação de licença: service@mathworks.com
- Compra, cotação e informações comerciais: info@mathworks.com

2 UTILIZANDO O MATLAB

O programa MATLAB pode ser executado em diversos sistemas operacionais, tais como: MS-DOS; Windows-95 ou superior; Unix; Linux; VAX/VMS; Solaris; X-Window System; Macintosh; e outros. Este curso está preparado para ser ministrado no sistema MS-DOS/Windows3.1x ou superior. No entanto, apenas o início do programa e as funções de baixo nível são afetados pelo tipo de máquina e sistema operacional utilizados.

2.1 Iniciar o Programa

O programa MATLAB pode ser iniciado com um duplo clique no ícone do MATLAB na área de trabalho do windows (*desktop*), conforme indica a Fig. 2.1.

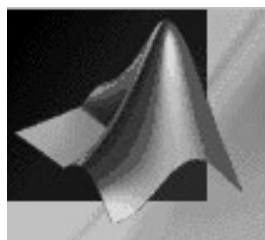


Fig. 2.1: Ícone do MATLAB.

Outra forma de iniciar o MATLAB é através do menu INICIAR do Windows95 ou superior. Seleciona-se: INICIAR / PROGRAMA / MATLAB.

Os arquivos executáveis do MATLAB estão no diretório de instalação do MATLAB, no subdiretório "BIN". O programa MATLAB é iniciado quando o arquivo MATLAB.exe, do subdiretório "BIN" é executado.

Ao iniciar, o MATLAB invoca o arquivo "MATLABrc", o qual é o arquivo de configuração inicial do MATLAB. Este arquivo pode estar no subdiretório "BIN", ou no subdiretório "LOCAL", ou ainda no subdiretório "...\\TOOLBOX\\LOCAL". Isto dependerá da versão do MATLAB utilizada, ou da forma como o programa foi instalado. Durante o processo de início do programa, o arquivo "MATLABrc" estabelece o tamanho padrão de figuras, configurações, caminhos e parâmetros de controle do MATLAB. Neste

arquivo pode-se incluir mensagens para os usuários, lembretes, ajustar configurações, etc.

Ao ser executado, o arquivo “MATLABrc” verifica se existe um arquivo denominado “startup.m” no caminho padrão do MATLAB. Este arquivo “startup.m” pode ser empregado pelo usuário para configurar um caminho de sua preferência e executar comandos de sua escolha, sempre que o programa MATLAB for iniciado. Todos os arquivos de comando do MATLAB (arquivos que contém comandos) têm extensão “m”. Portanto, daqui por diante as referências aos arquivos do MATLAB serão feitas apenas pelo nome, fica implícito que a extensão sempre será “m”.

Todos os arquivos de comando do MATLAB têm extensão “m”

Uma sessão do programa MATLAB pode ser terminada com o comando “quit”; com as teclas de atalho ctrl+q; selecionando-se a opção “Exit MATLAB” (“Sair do MATLAB”) no menu “File” (“Arquivo”); ou ainda, clicando-se no botão fechar na tela da área de trabalho do MATLAB.

Botão fechar do MATLAB:



2.2 Pasta de Trabalho

Uma boa prática de utilização do MATLAB é manter todos os arquivos criados pelo usuário em uma pasta (ou diretório) próprio do usuário. Caso esta prática não seja adotada todos os arquivos criados serão armazenados no diretório “BIN” do diretório de instalação do MATLAB. Com o passar do tempo, à medida em que aumentam os arquivos criados pelos usuários, torna-se impossível identificar-se quais arquivos são próprios do MATLAB, e quais arquivos são arquivos do usuário. Isto pode ser ainda mais prejudicial no caso de utilização do MATLAB em ambientes de rede LAN. Nestes casos, a desordem causada nos arquivos do diretório “BIN” pode requerer até mesmo uma nova instalação do programa.

Recomenda-se que o usuário crie a sua própria pasta de trabalho, fora da pasta de instalação do MATLAB (por exemplo, “c:\fulano\MATLAB\”). Ao iniciar o MATLAB, o usuário poderá redefinir a sua pasta de trabalho com o comando “cd”.

cd

Propósito: muda o diretório de trabalho

Sintaxe: `cd` ou `cd nome_do_diretorio`

Descrição: `cd` isoladamente imprime o caminho do diretório de trabalho na tela; `cd diretório` define a cadeia de caracteres (*string*) *nome_do_diretorio* como o novo diretório de trabalho.

`cd ..` sobe um nível na hierarquia da cadeia de diretórios (pasta)

Exemplo: `cd c:\usuario\marisa\matlab.`

Se o comando `cd` for parte integrante do arquivo “startup”, toda vez que o MATLAB for iniciado o diretório de trabalho será redefinido conforme a preferência do usuário.

2.3 Tela da Linha de Comando do MATLAB

A tela da linha de comando do MATLAB (ou *workspace*) é a tela na qual o usuário recebe o sinal de pronto (*prompt*) do sistema MATLAB, podendo executar comandos MATLAB, escrevendo-os na linha de comando (ou *command line*).

Na tela da linha de comando, o usuário fornece dados aos comandos do MATLAB, executa comandos, recebe os dados de saída do programa MATLAB, controla o fluxo de dados, inicia o modo de edição de arquivos, executa comandos gráficos, ou seja, interage com o programa MATLAB.

Ao criar funções, comandos e variáveis no programa MATLAB é importante estar atento para o fato de que este programa é sensível a caracteres maiúsculos e minúsculos, portanto a variável “xqualquer” é diferente da variável “Xqualquer” para o MATLAB.

O programa MATLAB é sensível a caracteres maiúsculos e minúsculos

2.4 Criação de Arquivos no MATLAB

Existem basicamente duas formas de trabalhar no sistema MATLAB: 1) executando-se comando na linha de comando; ou 2) criando-se um arquivo de comandos do MATLAB.

Para trabalhar na linha de comando do MATLAB basta digitar qualquer comando válido do MATLAB na linha de comando. Assim, é possível criar variáveis, executar comandos, visualizar gráficos, etc.

Muitas vezes é necessário executar um número grande de comandos, e repeti-los sistematicamente. Nestes casos, utilizam-se os arquivos de lote, ou arquivos de comandos MATLAB. Conforme já citada anteriormente, a extensão de arquivos de comando do MATLAB é “m”. O MATLAB busca primeiramente o arquivo de comandos no seu diretório de trabalho (pasta) e posteriormente no seu caminho (*path*). Portanto, é necessário que o arquivos MATLAB esteja no diretório de trabalho ou no caminho do MATLAB.

Os arquivos de comando do MATLAB devem estar no diretório de trabalho ou no caminho (*path*) para serem reconhecidos e executados

Os arquivos do MATLAB podem ser criados através de qualquer editor que salva textos no formato ASCII (por exemplo bloco de notas, *notepad*, write, word, etc). Uma vez editados os arquivos devem ser gravados com extensão “m” antes de serem executados.

Os arquivos MATLAB podem ser abertos ou criados através da opção arquivo (*file*), escolhendo-se a opção novo (*new*) ou abrir (*open*), conforme indicado na Fig. 2.2.

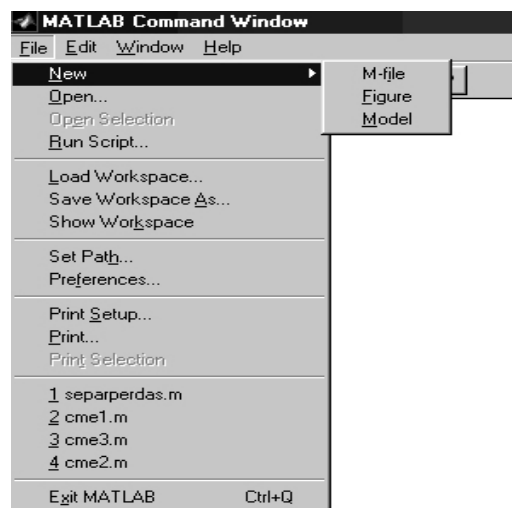


Fig. 2.2: Criação de arquivos de comandos no MATLAB.

O MATLAB já possui uma pré-definição de qual é o editor ASCII a ser utilizado na edição do arquivo de comandos. Esta pré-definição pode se modificada pelo próprio usuário do programa.

As versões mais recentes do MATLAB já possuem um editor ASCII próprio do MATLAB, desenvolvido pela The MathWorks Inc.

Exercício sugerido: criar um arquivo de comando denominado “caminho.m” que estabelece o diretório de trabalho do MATLAB.

2.5 Limitações do MATLAB

As principais limitações do programa MATLAB são: 1) a execução de algoritmos em MATLAB é mais lenta que em linguagens de programação (C, Fortran, LISP, etc); 2) alguns procedimentos gráficos e de interação com o usuário são restritos aos comandos do MATLAB; 3) não é possível gerar arquivos executáveis com o MATLAB, ou seja, um arquivo MATLAB só poderá ser executado no ambiente MATLAB; e 4) o MATLAB é limitado para processamento de expressões matemáticas analíticas.

Os principais concorrentes do MATLAB no mercado são o MATHEMATICA, o MATCAD e o MAPLE. Alguns destes programas apresentam melhor capacidade de processamento matemático analítico que o MATLAB, notadamente o MATCAD e o MATHEMATICA.

um arquivo MATLAB só poderá ser executado no ambiente MATLAB

3 OPERADORES, CONSTANTES E CARACTERES ESPECIAIS

3.1 Operadores aritméticos

Operador +

Propósito: executar soma de matrizes ou escalares.

Sintaxe: $v1+v2$ ou *plus(v1,v2)*.

Descrição: Invoca a função PLUS. Caso seja soma de matrizes, as dimensões das matrizes devem ser iguais. Escalares podem ser somados com qualquer tipo de variável.

Exemplos: $a+b$; *plus(a,b)*; $a+2$.

Operador -

Propósito: executar subtração de matrizes ou escalares.

Sintaxe: $v1-v2$ ou *minus(v1,v2)*.

Descrição: Invoca a função minus. Caso seja subtração de matrizes, as dimensões das matrizes devem ser iguais. Escalares podem ser subtraídos de qualquer tipo de variável.

Exemplos: $a-b$; *minus(a,b)*; $a-2$.

Operador *

Propósito: executar soma de matrizes ou escalares.

Sintaxe: $v1*v2$ ou *mtimes(v1,v2)*.

Descrição: Invoca a função mtimes. Caso seja multiplicação de matrizes, o número de colunas da matriz v1 deve ser igual ao número de linhas da matriz v2. Escalares podem multiplicar qualquer tipo de variável.

Exemplos: $a*b$ (neste caso, o número de colunas de a é igual ao número de linhas de b); *mtimes(a,b)*; $a*2$.

Operador ^ (circunflexo)

Propósito: executar potenciação de matrizes quadradas com expoente escalar ou de de escalares com matriz quadrada no expoente.

Sintaxe: $v1^v2$ ou *mpower(v1,v2)*.

Descrição: Invoca a função `mpower`. Caso $v1$ seja matriz quadrada e $v2$ seja um inteiro, é realizada a multiplicação sucessiva das matrizes. Caso $v1$ seja matriz e $v2$ seja um real, ou ainda se $v1$ é um escalar e $v2$ uma matriz, é realizado o cálculo através dos autovalores e dos autovetores da matriz. Se $v1$ e $v2$ forem ambas matrizes, ou ainda se $v1$ ou $v2$ forem matrizes não quadradas, então haverá erro.

Exemplos: a^b ou `mpower(a,b)` (neste caso, a é escalar e b é matriz quadrada; ou a é matriz quadrada e b é escalar; ou a e b são escalares).

Operador /

Propósito: executar divisão de escalares ou a multiplicação de uma matriz inversa por outra matriz (**importante: na matemática não existe divisão de matrizes**). É a chamada divisão direita.

Sintaxe: $v1/v2$ ou `mrdivide(v1,v2)`.

Descrição: Invoca a função `mrdivide`. Esta operação é equivalente a $v1*v2^{-1}$ (sendo $v2^{-1}$ a matriz inversa da matriz $v2$). Para que esta operação seja possível é necessário que $v2$ seja inversível, ou seja, matriz não-singular (matriz quadrada com determinante não-nulo).

Exemplos: a/b ou `mrdivide(a,b)`.

Operador \

Propósito: executar divisão de escalares ou a multiplicação de uma matriz inversa por outra matriz. É a chamada divisão esquerda.

Sintaxe: $v1\backslash v2$ ou `mldivide(v1,v2)`

Descrição: Invoca a função `mldivide`. Esta operação é equivalente a $v1^{-1}*v2$ (sendo $v1^{-1}$ a matriz inversa da matriz $v1$). Para que esta operação seja possível é necessário que $v1$ seja inversível, ou seja, matriz não-singular (matriz quadrada com determinante não-nulo). **Observe que na álgebra matricial a multiplicação não é comutativa, ou seja $a*b$ é diferente de $b*a$, portanto a/b equivale a $a*b^{-1}$, que por sua vez é diferente de $b\backslash a$, equivalente a $b^{-1}*a$.**

O operador `\` executa a fatoração QR da matriz A ao resolver o sistema $Ax = b$, para A de ordem $m \times n$, para m não necessariamente igual a n .

Exemplos: `a\b` ou `mldivide(a,b)`.

Na matemática não existe operação de divisão com matrizes. Os operadores `/` e `\` são apenas símbolos.

A multiplicação na álgebra matricial não é comutativa, ou seja, $a*b$ é diferente de $b*a$.

Operador `'`

Propósito: executar a transposição de matrizes ou a declaração de uma cadeia de caracteres (*string*).

Sintaxe: `v1'` ou `v2='bom dia'`

Descrição: `v1'` produz a matriz $v1'^T$ (matriz transposta **conjugada** da matriz `v1`, troca de linhas por colunas dos conjugados dos números complexos da matriz `v1`). `v2='bom dia'` cria uma variável do tipo *string*, onde cada caractere ocupa 2 bytes na memória.

Exemplos: `a'*b` ou `salutation='Hello!'`.

Cada caractere ocupa 2 bytes de memória RAM nas variáveis tipo *string*. Cada variável real (*float*) ou inteira ocupa 8 bytes de memória. Em uma matriz de *strings*, a área de memória ocupada em bytes, 2 vezes, o número de caracteres. Em uma matriz de reais ou inteiros, a área de memória ocupada, em bytes, é 8 vezes o número de elementos da matriz.

3.2 Caracteres Especiais

character .

Propósito: executar operações elemento-a-elemento (operações algébricas escalares (`/`, `\`, `^`, `*`, `'`), não-matriciais); indexar campos em estruturas; ou ponto decimal.

Sintaxe: `a.*a`; `a./b`; `c.\f`; `x.^2`; `x.^y` para operações algébricas; ou `est.campo`

Descrição: `a.*b` multiplica o elemento $a(i,j)$ pelo elemento $b(i,j)$. `a.'`

$v1.opv2$ executa a operação op de cada elemento de $v1$ com seu elemento correspondente em $v2$. No caso de operação entre matrizes, as matrizes devem possuir exatamente as mesmas dimensões. A operação com escalar pode ser realizada com matriz de qualquer dimensão.

$a.'$ é a matriz transposta **não conjugada** da matriz a .

$est.camp$ seleciona o campo 'camp' da estrutura 'est'.

Exemplos: $2.^x$ é diferente de 2^x ; $x.*x$ é diferente de x^x . $auto.peugeot$ seleciona o campo peugeot da estrutura auto.

O caractere `.` produz a operação elemento-a-elemento entre matrizes com exatamente a mesma dimensão ou entre escalares e matrizes.

caracter :

Propósito: Criar varredura limitada pelos números à direita e à esquerda de `:`.

Sintaxe: $j:k; j:d:k$.

Descrição: $j:k$ é o mesmo que $[j \ j+1 \ j+2 \ \dots \ k]$, $j:k$ é vazio se $k < j$. $j:d:k$ é o mesmo que $[j \ j+d \ j+2*d \ \dots \ k]$, $j:d:k$ é vazio se $k < j$. d pode ser negativo, mas neste caso k deve ser menor que j .

Exemplos: $1:10$; $0:5:100$; $100:-5:0$.

caracter =

Propósito: executa atribuições

Exemplos: $x=[1 \ 2 \ 3; 4.5 \ 6 \ 7.5];$

caracter ,

Propósito: separar elementos em uma matriz; indexar matrizes; separar argumento de uma função; separar comando em uma linha com vários comandos.

Exemplos: $2*a(2,3)$; $plot(x,y)$; $x(i,j)$; $mat=[a, b, c]$; $clear$, $disp('memória limpa')$.

caracter ;

Propósito: usado em declaração de matrizes para separar linhas; ou usado para omitir a apresentação do resultado de uma operação na tela.

Exemplos: $x=[1 \ 2 \ 3; 9 \ 8 \ 7]$; $a=2+3$;

character !

Propósito: invoca comando do sistema operacional.

Exemplos: ! mkdir novo.

character %

Propósito: torna o texto à direita de % um comentário.

Exemplos: x=[1 2 3; 4.5 6 7.5]; %cria matriz x.

caracteres ..

Propósito: representa o primeiro diretório na hierarquia de diretórios.

Exemplos: cd .. sobe um diretório na hierarquia.

caracteres ... (três ou mais pontos)

Propósito: representa continuação de linha.

Exemplos: a=2+...

3 é o mesmo que a=2+3.

caracteres ()

Propósito: indicam precedência em operações matemáticas ou indexam matrizes.

Exemplos: (a+b)^c, a soma é realizada antes da potenciação; x(3) é o terceiro elemento do vetor x; x([1 2 3]) são os três primeiros elementos do vetor x.

character { }

Propósito: são utilizadas para formar estruturas (conjunto de células), onde cada célula pode ser outra estrutura, uma matriz ou uma string. Funciona como os [], porém seus elementos podem ser outras matrizes.

Podem também ser utilizadas para indexar estruturas e dar acesso aos seus campos.

Exemplo: l={'isto' 'é' 'uma' 'matriz linha:' [1 2 24]}, l é uma estrutura com 5 campos.

3.3 Operadores Lógicos**caracteres & | ~**

Propósito: realizam as operações lógicas e, ou e negação com os operadores &, | e ~, respectivamente.

Sintaxe: a&b; A|c; ~d; os operandos a,b,c,d,A podem ser números reais (float), complexos (utiliza o módulo) ou strings (utiliza o valor da tabela ASCII). No MATLAB, 0 é considerado falso, e qualquer outro valor não-nulo é verdadeiro. O resultado destas operações será verdadeiro (1) ou falso (0).

Exemplos: 5&-3 resulta em 1; ~10 resulta em 0; 'alfa'|0 resulta em [1 1 1 1].

3.4 Operadores relacionais

caracteres < <= > >= == ~=

Propósito: realizam as operações de comparação de valores lógicos do tipo menor, menor igual, maior, maior igual, igual e diferente com os operadores < <= > >= == ~=, respectivamente. O resultado é um valor lógico verdadeiro (1) ou falso (0).

Sintaxe: v1opv2 testa o valor lógico v1 em relação ao valor lógico v2, com a operação relacional op.

Exemplos: 3<4 resulta em 1; 3==4 resulta em 0; a>=b verifica se a é maior ou igual a b, resultando verdadeiro (1) ou falso (0).

3.5 Constantes ou Variáveis

Alguns caracteres são constantes definidos pelo MATLAB. Estes valores **podem ser alterados temporariamente** através de atribuição.

Constante pi

Definição: é a constante π (3,1416...)

Constantes i e j

Propósito: caracteres reservados com a unidade imaginária $\sqrt{-1}$.

Sintaxe: número seguido de i ou j, ou multiplicado por i ou j.

Exemplos: 5+3i ou 5+3*i; 5+3j ou 5+ 3*j.

ans

Definição: é uma variável cujo valor é o resultado da última operação aritmética realizada pelo MATLAB. Operações de atribuição não são armazenadas em ans. Abreviatura para *answer*.

3.5.1 Inf

Definição: é o resultado de uma operação aritmética cujo valor ultrapassa a maior capacidade de representação do MATLAB. Por exemplo, divisão por zero, ou $1.7977e+308 \wedge 1.7977e+308$. Abreviatura para *infinite*.

3.5.2 NaN

Definição: é o resultado de uma operação aritmética indeterminada. Por exemplo, 0/0 Abreviatura para *Not a Number*.

3.5.3 iieee

Definição: verifica se o computador em que o MATLAB está sendo executado possui aritmética padrão IEEE. Os computadores IBM-PC e compatíveis, Macintosh, estações UNIX e Linux possuem aritmética padrão IEEE. Os computadores VAX e Cray normalmente não utilizam aritmética IEEE. A função retorna 1 se o computador tem aritmética IEEE; caso contrário, retorna 0.

Examine também: eps; isnan; isinf; isfinite; flops.

3.6 Recursos de Apoio da Tela de Comando do MATLAB

A tela de comando do programa MATLAB apresenta uma série de recursos que podem auxiliar o usuário. Entre estes recursos destacam-se:

3.6.1 Menu File

Neste Menu é possível abrir um arquivo MATLAB já existente (open); iniciar um novo arquivo com extensão m (new), através do editor ASCII do MATLAB; executar um arquivo com comandos MATLAB (run script); imprimir arquivos e seleções (print e print selection); carregar variáveis de um arquivo (load workspace); salvar variáveis do workspace (área de trabalho) em um arquivo; monitorar a memória da área de trabalho (show workspace); redefinir o caminho de diretórios do MATLAB (set path); alterar as configurações da área de trabalho, tais como cor de fundo, formato de variáveis, etc (preferences).

3.6.2 Menu Edit

Aqui é possível cortar (cut), copiar (copy), colar (paste), limpar a tela da sessão de trabalho (clear session).

Botões de Auxílio:



1º botão da figura: Voltar - cancela a última alteração;

2º botão da figura: Workspace browser - monitora as variáveis e a memória da área de trabalho;

3º botão da figura: Path browser - monitora e redefine os diretórios do caminho do MATLAB;

4º botão da figura: SIMULINK - invoca o programa SIMULINK;

5º Ajuda em tempo real - fornece auxílio para todos os comandos do MATLAB (invoca a função helpwin).

A ordem destes botões pode ser alterada pelo usuário do MATLAB.

4 COMANDOS DE PROPÓSITO GERAL

4.1 Comandos de Informação

help

Propósito: Ajuda em tempo real (*on-line*). O texto é apresentado na tela da linha de comando.

Sintaxe: help nome-da-função.

Descrição: fornece informações sobre a função especificada pelo nome-da-função.

Exemplo: help plot.

helpwin

Propósito: Ajuda em tempo real (*on-line*). O texto é apresentado em uma janela de navegação separada.

Sintaxe: helpwin.

Descrição: é aberta uma janela com opções para escolha do nome e categoria da função.

ver

Propósito: informa as versões do MATLAB, SIMULINK e toolboxes.

Sintaxe: ver

version

Propósito: informa as versões do MATLAB

Sintaxe: version

demo

Propósito: executa programas de demonstração desenvolvidos em ambiente MATLAB.

Sintaxe: demo.

whatsnew

Propósito: Ajuda em tempo real (*on-line*). O texto é apresentado em uma janela de navegação separada.

Sintaxe: `whatsnew tipo-de-função`.

Descrição: apresenta as atualizações dos comandos tipo-de-função na tela da linha de comando.

Exemplo: `whatsnew MATLAB`; `whatsnew general`.

bench

Propósito: executa um conjunto de funções MATLAB com a finalidade de avaliar o desempenho do processador atual utilizado em relação a outros processadores.

Sintaxe: `bench`.

4.2 Comandos de Operação com a Memória da Área de Trabalho (workspace)

who

Propósito: lista as variáveis atuais.

Sintaxe: `who`.

whos

Propósito: lista as variáveis atuais, suas dimensões e memória que cada uma ocupa.

Sintaxe: `whos`.

clear

Propósito: limpa completamente a memória, função ou variável.

Sintaxe: `clear`; `clear arg`.

Descrição: `clear` limpa toda a memória da área de trabalho; `clear arg` exclui a função ou variável `arg` da memória da área de trabalho.

Exemplo: `clear`; `clear x1`.

pack

Propósito: desfragmenta a memória da área de trabalho.

Descrição: `pack` reúne todas as variáveis em um segmento ou páginas adjacentes na memória RAM com a finalidade de otimizar o uso da memória.

Sintaxe: `pack`.

save

Propósito: salva as variáveis da memória da área de trabalho em arquivo.

Sintaxe: save fname; save fname v1 v2 ... vn.

Descrição: save fname salva todas as variáveis da área de trabalho no arquivo cujo nome e caminho são descritos em fname. save fname v1 v2 ... vn salva apenas as variáveis v1, v2 e vn.

Exemplo: save a:\hoje; save c:\MATLAB\arq x y.

load

Propósito: carrega as variáveis de um arquivo para a memória da área de trabalho.

Sintaxe: load fname; load fname v1 v2 ... vn.

Descrição: load fname carrega todas as variáveis do arquivo cujo nome e caminho são descritos em fname para a memória da área de trabalho. load fname v1 v2 ... vn carrega apenas as variáveis v1, v2 e vn.

Exemplo: load a:\hoje; load c:\MATLAB\arq x y.

quit

Propósito: encerra a sessão de trabalho do MATLAB

Sintaxe: quit.

4.3 Comandos de Operação com Funções e Comandos

what

Propósito: lista os nomes de arquivos de funções e comandos de um grupo específico do MATLAB.

Sintaxe: what nome; var=what('nome').

Descrição: what nome lista todos os arquivos e funções do grupo descrito por nome. var=what('nome') cria uma estrutura cujos campos descrevem os diversos arquivos relacionados com o grupo definido por nome.

Exemplo: what general; w=what('signal').

type

Propósito: lista o conteúdo de um arquivo de comandos do MATLAB.

Sintaxe: type arq.

Exemplo: `type multi.m`; `type c:\MATLAB\marisa\xtudo.m`.

edit

Propósito: edita arquivo do MATLAB

Sintaxe: `edit`; `edit arq`.

Descrição: somente `edit` inicia a edição de um novo arquivo; `edit arq` abre a edição de um arquivo `arq` já existente.

Exemplo: `edit`; `edit figs.m`.

lookfor

Propósito: busca a palavra-chave em todos os arquivos contidos no caminho do MATLAB.

Sintaxe: `lookfor palavra`.

Descrição: procura, em todos os arquivos do caminho, a palavra-chave contida na string `palavra`, e lista o nome dos arquivos nos quais existe a palavra-chave.

Exemplo: `lookfor matrix`.

Examine também os comandos: `which`; `pcode`; `inmem`; `mex`.

4.4 Comandos de Operação com o Caminho de Diretórios

path

Propósito: verifica ou define o caminho de diretórios.

Sintaxe: `path`; `path=caminho`; `v=path`.

Descrição: `path` lista na tela da linha de comando o caminho atual.

`path=caminho` define o caminho atual como sendo a string contida em `caminho`.

`v=path` carrega a string correspondente ao caminho atual em `v`.

Exemplo: `path`; `path='c:\usuario\MATLAB'`; `p1=path`.

addpath

Propósito: adiciona diretórios ao caminho.

Sintaxe: `addpath nome`.

Descrição: adiciona o diretório descrito na string `nome` ao caminho atual.

Exemplo: `addpath 'c:\user\pink'`.

rmpath

Propósito: remove diretórios do caminho.

Sintaxe: rmpath nome.

Descrição: remove o diretório descrito na string nome ao caminho atual.

Exemplo: rmpath 'c:\user\pink'.

Examine também os comandos: editpath.

4.5 Comandos de Operação com a Janela de Comando

echo

Propósito: habilita ou inibe a apresentação dos resultados de comando de um arquivo MATLAB na linha de comando.

Sintaxe: echo on; echo off.

Descrição: echo on habilita a apresentação e echo off desabilita.

more

Propósito: habilita ou inibe a paginação do texto na tela de comandos.

Sintaxe: more on; more off.

Descrição: echo on habilita a paginação e echo off desabilita.

diary

Propósito: salva todo o texto da sessão de trabalho em arquivo

Sintaxe: diary on; diary off; diary(nome).

Descrição: após diary on todo texto que aparece na janela de comando é também enviado para um arquivo nomeado automaticamente pelo MATLAB. diary off interrompe o salvamento em arquivo. diary(nome) salva o texto da janela de comando no arquivo cujo nome é descrito pela string nome.

Exemplo: diary on; diary off; diary('31-8-00').

format

Propósito: determina o formato no qual os dados serão expressos na janela de comando.

Sintaxe: format opção.

Descrição: expressa os dados conforme opção. opção: 1) short ou em branco: formato ponto decimal com 5 dígitos; 2) long: formato ponto decimal com 15 dígitos; 3) short e: formato ponto flutuante com 5 dígitos; 4) long: formato ponto flutuante com 15 dígitos; 5) short g: o MATLAB seleciona entre o formato ponto decimal com 5 dígitos ou ponto flutuante com 5 dígitos; 6) long g: o MATLAB seleciona entre o formato ponto decimal com 15 dígitos ou ponto flutuante com 15 dígitos; 7) hex: hexadecimal; 8) +: apresenta o sinal + ou – para os valores; 9) bank: formato bancário com \$ e centavos; 10) rat: aproxima para a fração mais próxima do valor a ser expresso.

4.6 Comandos de Operação com o Sistema Operacional

cd

Propósito: altera o diretório de trabalho atual.

Sintaxe: cd diretório.

Exemplo: cd c:\MATLAB\usuario.

pwd

Propósito: exibe o diretório de trabalho atual.

Sintaxe: pwd.

dir

Propósito: lista os arquivos do diretório atual.

Sintaxe: dir.

delete

Propósito: exclui arquivo do diretório.

Sintaxe: delete arq.

Exemplo: delete tarefa.m.

dos

Propósito: executa comando do MS-DOS.

Sintaxe: dos comando.

Exemplo: dos copy a:\antigo.m a:\novo.m.

computer

Propósito: definir o tipo de computador em uso.

Sintaxe: computer.

Descrição: retorna uma string com o tipo do computador em uso.

web

Propósito: abrir página na internet ou arquivo html com o browser padrão do sistema.

Sintaxe: web arquivo; web endereço.

Exemplo: web c:\paginas\info.htm; web http://www.mathworks.com.

Examine também os comandos: getenv; unix; vms; realmax; realmin; clock; cputime; date; etime; tic; toc.

5 COMANDOS BÁSICOS PARA MANIPULAÇÃO DE MATRIZES

zeros

Propósito: cria uma matriz esparsa (composta de zeros).

Sintaxe: `zeros(n)`; `zeros(n,m)`; `zeros(size(a))`;

Descrição: `zeros(n)` cria uma matriz quadrada composta de zeros; `zeros(n,m)` cria uma matriz n por m composta de zeros; `zeros(size(a))` cria um matriz com as mesmas dimensões da matriz a, composta de zeros.

Exemplo: `zeros(4)`; `zeros(2,3)`; `zeros(size(a))`.

ones

Propósito: cria uma matriz com elementos iguais a 1.

Sintaxe: `ones(n)`; `ones(n,m)`; `ones(size(a))`;

Descrição: `ones(n)` cria uma matriz quadrada composta de elementos iguais a 1; `ones(n,m)` cria uma matriz n por m composta de elementos iguais a 1; `ones(size(a))` cria um matriz com as mesmas dimensões da matriz a, composta de elementos iguais a 1.

Exemplo: `ones(4)`; `ones(2,3)`; `ones(size(a))`.

eye

Propósito: cria uma matriz identidade ou matriz com diagonal 1.

Sintaxe: `eye(n)`; `eye(n,m)`; `ones(size(a))`;

Descrição: `eye(n)` cria uma matriz quadrada composta de elementos iguais a 1 na diagonal principal e demais elementos nulos; `eye(n,m)` cria uma matriz n por m composta de elementos iguais a 1 na diagonal principal e demais elementos nulos; `eye(size(a))` cria um matriz com as mesmas dimensões da matriz a, composta de elementos iguais a 1 na diagonal principal e demais elementos nulos.

Exemplo: `eye(4)`; `eye(2,3)`; `eye(size(a))`.

rand

Propósito: cria uma matriz com elementos gerados aleatoriamente.

Sintaxe: `rand(n)`; `rand(n,m)`; `rand(size(a))`;

Descrição: `rand(n)` cria uma matriz quadrada composta de elementos gerados aleatoriamente; `rand(n,m)` cria uma matriz n por m composta de elementos gerados aleatoriamente; `rand(size(a))` cria uma matriz com as mesmas dimensões da matriz a, composta de elementos gerados aleatoriamente.

Exemplo: `rand(4)`; `rand(2,3)`; `rand(size(a))`.

randn

Propósito: o mesmo de `rand`, porém neste caso a distribuição segue o padrão normal, enquanto `rand` segue padrão uniforme.

diag

Propósito: extrai diagonais de matrizes e cria um vetor com estes elementos.

Sintaxe: `diag(m,k)`; `diag(m)`;

Descrição: `diag(m)` cria um vetor com a diagonal principal da matriz m. `diag(m,k)` cria um vetor com a k-ésima diagonal acima ($k > 0$) ou abaixo ($k < 0$) da diagonal principal. `diag(m)` equivale a `diag(m,0)`.

Exemplo: `diag(a)`; `diag(a,1)`.

linspace

Propósito: cria um vetor com elementos linearmente espaçados.

Sintaxe: `linspace(x1,x2)`; `linspace(x1,x2,N)`.

Descrição: `linspace(x1,x2)` gera um vetor com 100 pontos igualmente espaçados entre x_1 e x_2 . `linspace(x1,x2,N)` gera um vetor com N pontos igualmente espaçados entre x_1 e x_2 .

Exemplo: `linspace(5,25)`; `linspace(10,1000,200)`.

logspace

Propósito: cria um vetor com elementos logaritmicamente espaçados.

Sintaxe: `logspace(x1,x2)`; `logspace(x1,x2,N)`.

Descrição: `logspace(x1,x2)` gera um vetor com 50 pontos logaritmicamente espaçados entre 10^{x_1} e 10^{x_2} . `logspace(x1,x2,N)` gera um vetor com N pontos logaritmicamente espaçados entre 10^{x_1} e 10^{x_2} .

Exemplo: `logspace(-1,100)`; `logspace(10,1000,200)`.

size

Propósito: retorna a dimensão de uma matriz ou estrutura.

Sintaxe: `size(a)`.

Descrição: retorna um vetor com as dimensões da matriz ou estrutura `a`.

length

Propósito: retorna o comprimento de um vetor.

Sintaxe: `size(a)`.

Descrição: retorna o comprimento do vetor `a`. Se `a` for matriz, `size(a)` retorna a maior dimensão de `a`.

tril ou triu

Propósito: extrai a matriz triangular inferior (`tril`) ou a matriz triangular superior (`triu`).

Sintaxe: `tril(a)` ou `triu(a)`; `tril(a,k)` ou `triu(a,k)`.

Descrição: extrai a matriz triangular superior (`triu`) ou inferior (`tril`) à diagonal principal. Extrai a matriz triangular superior (`triu`) ou inferior (`tril`) à k -ésima linha acima ($k > 0$) ou abaixo ($k < 0$) da diagonal principal. `tril(a)` equivale a `tril(a,0)`, e `triu(a)` equivale a `triu(a,0)`.

Exemplos: `triu(x)`; `tril(x,2)`.

Examine também os comandos: `meshgrid`; `::`; `ndims`; `reshape`; `find`; `end`; `sub2ind`; `ind2sub`; `fliplr`; `flipup`; `flipdim`; `rot90`.

6 FUNÇÕES MATEMÁTICAS ELEMENTARES

abs

Propósito: extrai o módulo (valor absoluto) de um valor algébrico real ou complexo.

Sintaxe: `abs(arg)`.

acos

Propósito: determina o valor inverso (arco) do cosseno em radianos.

Sintaxe: `acos(arg)`.

acosh

Propósito: determina o valor inverso (arco) do cosseno hiperbólico em radianos.

Sintaxe: `acosh(arg)`.

angle

Propósito: determina o ângulo em radianos de um número complexo.

Sintaxe: `angle(complexo)`.

asin

Propósito: determina o valor inverso (arco) do seno em radianos.

Sintaxe: `asin(arg)`.

asinh

Propósito: determina o valor inverso (arco) do asinh hiperbólico em radianos.

Sintaxe: `asinh(arg)`.

atan

Propósito: determina o valor inverso (arco) da tangente em radianos.

Sintaxe: `atan(arg)`.

atanh

Propósito: determina o valor inverso (arco) da tangente hiperbólico em radianos.

Sintaxe: `atanh(arg)`.

atan2

Propósito: determina o valor inverso (arco) da tangente em radianos do ângulo de número complexo, nos **quatro quadrantes**.

Sintaxe: atan2(parte-imag,parte-real)

ceil

Propósito: arredonda para o primeiro inteiro algebricamente superior.

Sintaxe: ceil(arg).

fix

Propósito: arredonda para o inteiro mais próximo de zero.

Sintaxe: fix(arg).

round

Propósito: arredonda para o inteiro mais próximo (arredondamento matemático convencional).

Sintaxe: round(arg).

sign

Propósito: retorna 1 se o argumento é positivo e 0 se o argumento é negado.

Sintaxe: sign(arg).

imag

Propósito: extrai a parte imaginária de um complexo.

Sintaxe: imag(complexo).

real

Propósito: extrai a parte real de um complexo.

Sintaxe: real(complexo).

conj

Propósito: extrai o conjugado de um complexo.

Sintaxe: conj(complexo).

cos

Propósito: calcula o cosseno de um arco em radianos.

Sintaxe: cos(arco).

sin

Propósito: calcula o seno de um arco em radianos.

Sintaxe: sin(arco).

tan

Propósito: calcula a tangente de um arco em radianos.

Sintaxe: tan(arco).

cosh

Propósito: calcula o cosseno hiperbólico de um arco em radianos.

Sintaxe: cosh(arco).

sinh

Propósito: calcula o seno hiperbólico de um arco em radianos.

Sintaxe: sinh(arco).

tanh

Propósito: calcula a tangente hiperbólica de um arco em radianos.

Sintaxe: tanh(arco).

exp

Propósito: calcula o valor de e^{arg} .

Sintaxe: exp(arg).

pow2

Propósito: calcula o valor de 2^{arg} .

Sintaxe: pow2(arg).

sqrt

Propósito: calcula o valor da raiz quadrada de arg.

Sintaxe: sqrt(arg).

log

Propósito: calcula o valor do logaritmo na base natural e (ln).

Sintaxe: log(arg).

log10

Propósito: calcula o valor do logaritmo na base 10.

Sintaxe: log10(arg).

log2

Propósito: calcula o valor do logaritmo na base 2.

Sintaxe: log2(arg).

rem

Propósito: retorna o resto inteiro de uma divisão de x por y.

Sintaxe: rem(x,y).

Examine também os comandos: sec; sech; asec; asech; csc; csch; acsc; acsch; cot; coth; acot; acoth; nextpow2; unwrap; isreal; cplxpair; mod.

7 COMANDOS DE CONTROLE GRÁFICO

7.1 Gráficos Bidimensionais

plot

Propósito: imprime o vetor em gráfico bidimensional.

Sintaxe: `plot(x,y,'cst')`

Descrição: imprime o vetor y na ordenada em relação ao vetor x na abcissa. x e y devem obrigatoriamente ter as mesmas dimensões. A string 'cst' permite selecionar a cor ($c=ymrcrgbwk$), o tipo de símbolo ($s=*.ox+sdv^{<>ph}$), e o tipo de linha ($t=--:--$).

Exemplo: `plot(x,sin(x),'m*-.)` plota seno de x em cor magenta, com símbolos * para cada ponto e linha tracejada e pontilhada.

No comando plot os vetores de ordenada e abscissa devem ter as mesmas dimensões.

subplot

Propósito: divide uma figura em vários gráficos.

Sintaxe: `subplot(n,m,p)`

Descrição: `subplot(n,m,p)` divide a figura em uma matriz $n \times m$ de gráficos e seleciona o p -ésimo gráfico. p é contado da esquerda para a direita, continuando na linha de baixo.

Exemplo: `subplot(3,3,5)` divide a figura em 9 gráficos, distribuídos em 3 linhas e 3 colunas, e seleciona o gráfico central.

O comando subplot apenas seleciona um dos gráficos da matriz $n \times m$, mas o comando plot é que executa a impressão.

loglog

Propósito: o mesmo que plot, porém agora as escalas de ordenadas e abscissas são do tipo logarítmicas.

Sintaxe: `loglog(x,y,'cst')`

semilogx

Propósito: o mesmo que plot, porém agora a escala da abscissa é do tipo logarítmica.

Sintaxe: `semilogx(x,y,'cst')`

semilogy

Propósito: o mesmo que plot, porém agora a escala da ordenada é do tipo logarítmica.

Sintaxe: `semilogy(x,y,'cst')`

polar

Propósito: desenha gráfico em formato polar.

Sintaxe: `polar(ang,raio,'cst')`

Descrição: imprime cada elemento do vetor raio em relação ao elemento correspondente do vetor ang (ângulo). A string 'cst' permite selecionar a cor (c=ymrcrgbwk), o tipo de símbolo (s=*.ox+sdv^<>ph), e o tipo de linha (t=--:--).
Ex.: `polar((linspace(0:2*pi),4*ones(1,100)))` plota uma circunferência de raio 4.

axis

Propósito: controla a aparência das escalas da abscissa e da ordenada.

Sintaxe: `axis([xmin xmax ymin ymax])`.

Descrição: ajusta a escala da abscissa para valores entre xmin e xmax, e a escala da ordenada para valores entre ymin e ymax.

Exemplo: `axis([-3 4 0 10])` desenha o gráfico com escalas de -3 a 4 para o eixo da abscissa e 0 a 10 para o eixo da ordenada.

grid

Propósito: traça uma grade sobre o plot atual.

Sintaxe: `grid`.

hold

Propósito: retém o estado do plot atual de modo a permitir que o próximo gráfico seja impresso sobre o atual. `hold on` liga o estado do comando e `hold off` desliga o estado do comando.

Sintaxe: `hold`; `hold on`; `hold off`.

title

Propósito: Cria texto de título no plot atual.

Sintaxe: title('texto').

xlabel

Propósito: Cria texto para o eixo da abcissa.

Sintaxe: xlabel('texto').

ylabel

Propósito: Cria texto para o eixo da ordenada.

Sintaxe: ylabel('texto').

gtext

Propósito: Cria texto para ser posicionado com o mouse.

Sintaxe: gtext('texto').

Examine também os comandos: plotyy; axes; box; legend; text; print; printopt; orient.

7.2 Gráficos Tridimensionais

As funções de manipulação e controle de gráficos em três eixos e funções gráficas mais avançadas são dadas no quadro a seguir:

Examine os comandos: plot3d; fill3; contour; contour3; clabel; contourc; pcolor; quiver; image; mesh; meshc; meshz; surf; surfc; surfl; waterfall; slice; view; viewmtx; hidden; shading axis; caxis; colormap; title; xlabel; ylabel; zlabel; text; gtext; grid; cylinder; sphere; figure; gcf; clf close; gca; caxis; hold; line; patch; text; surface; image; uicontrol; uimenu; set; ger; reset; delete; drawnow; print; printop; moviein; getframe; movie; ginput; ishould.

8 OUTROS COMANDOS

8.1 Funções Lógicas

exist

Propósito: verifica se uma variável ou função existe no caminho do MATLAB.

Sintaxe: `exist(arg)`.

Descrição: `exist(arg)` retorna 1 se existir a variável ou função `arg`; caso contrário retorna 0.

Exemplo: `exist(var1)`.

any

Propósito: verifica se um vetor possui um elemento diferente de zero.

Sintaxe: `any(vet)`.

Descrição: `any(vet)` retorna 1 se existir um único elemento diferente de zero no vetor `vet`; caso contrário retorna 0.

Exemplo: `any(nomes)`.

all

Propósito: verifica se um vetor possui todos elementos diferentes de zero.

Sintaxe: `all(vet)`.

Descrição: `all(vet)` retorna 1 se todos os elementos de `vet` forem diferentes de zero; caso contrário retorna 0.

Exemplo: `all(nomes)`.

find

Propósito: encontra os índices dos elementos não-nulos de uma matriz.

Sintaxe: `find(mat)`.

Exemplo: `find(rand(3))`.

isnan

Propósito: verifica se há elementos do tipo NaN em uma matriz.

Sintaxe: `isnan(mat)`.

Descrição: `isnan(vet)` retorna uma matriz onde os elementos são 1 na posição onde há um elemento NaN na matriz original e 0 nas demais posições.

Exemplo: `isnan(mat)`.

isinf

Propósito: verifica se há elementos do tipo inf em uma matriz.

Sintaxe: `isinf(mat)`.

Descrição: `isinf(mat)` retorna uma matriz onde os elementos são 1 na posição onde há um elemento inf na matriz original e 0 nas demais posições.

Exemplo: `isinf(mat)`.

finite

Propósito: verifica se há elementos finitos em uma matriz

Sintaxe: `finite(mat)`.

Descrição: `finite(mat)` retorna uma matriz onde os elementos são 1 na posição onde há um elemento diferente de NaN ou de Inf na matriz original e 0 nas demais posições.

Exemplo: `finite(mat)`.

ischar

Propósito: verifica se o argumento é uma cadeia de caracteres (string).

Sintaxe: `ischar(arg)`.

Descrição: `ischar(arg)` retorna 1 se arg for uma cadeia de caracteres; caso contrário retorna 0.

Exemplo: `ischar('nome')` retornará 1.

Examine também os comandos: `isieee`; `isempty`; `issparse`.

8.2 Funções de Controle de Fluxo de Programação

if, else, elseif e end

Propósito: testar condição e determinar sentenças a serem executadas.

Sintaxe:


```

IF exp1
    stat1
ELSEIF exp2
    stat2
ELSE
    stat3
END

```

Descrição: testa exp1. Se exp1 for verdadeira executa as sentenças em stat1 e vai para end; caso contrário, testa exp2. Se exp2 for verdadeira executa as sentenças em stat2 e vai para end; caso contrário executa stat3.

Exemplo:

```

if I == J
    A(I,J) = 2;
elseif abs(I-J) == 1
    A(I,J) = -1;
else
    A(I,J) = 0;
end

```

for, end

Propósito: cria, incrementa e testa índice de controle para execução de sentenças.

Sintaxe: FOR var = expr, stat1, stat2,..., statn END

Descrição: enquanto a variável var for inferior ao limite estabelecido, as sentenças stat1, stat2 até statn serão executadas.

Exemplo:

```

FOR I = 1:N,
    FOR J = 1:N,
        A(I,J) = 1/(I+J-1);
    END
END

```

while, end

Propósito: testa a expressão de controle para execução de outras sentenças.

Sintaxe:

```
WHILE exp
  stats
END
```

Descrição: enquanto a expressão `exp` for verdadeira, as sentenças `stats` serão executadas.

Exemplo:

```
E = 0*A; F = E + eye(size(E)); N = 1;
while norm(E+F-E,1) > 0,
E = E + F;
F = A*F/N;
N = N + 1;
end
```

switch, end

Propósito: testa expressão de controle e seleciona um caso para executar.

Sintaxe:

```
SWITCH switch_expr
  CASE case_expr,
    stat1, ..., stat1n
  CASE {case_expr1, case_expr2, case_expr3,...}
    stat2, ..., stat2n
  ...
  OTHERWISE,
    stat3, ..., stat3n
END
```

Descrição: Se `switch_expr` for igual a `case_expr`, então as sentenças `stat1` até `stat1n` serão executadas, e terminado o comando. Se `switch_expr` for igual a `case_expr1` ou `case_expr2` ou `case_expr3` ..., então as sentenças `stat2` até `stat2n` serão executadas, e terminado o comando. Se `switch_expr` for diferente de todos as `case_expr`'s anteriores, então as sentenças `stat3` até `stat3n` serão executadas, e terminado o comando.

Exemplo:

```
switch lower(METHOD)
    case {'linear','bilinear'}, disp('Method is linear')
    case 'cubic', disp('Method is cubic')
    case 'nearest', disp('Method is nearest')
    otherwise, disp('Unknown method.')
end
```

break

Propósito: encerra a execução de comandos for ou while quando são encontrados.

Sintaxe: break.

input

Propósito: exibe mensagem na tela e dá ao usuário um prompt para entrada de dados.

Sintaxe: input('texto'); input('texto','s').

Descrição: input('texto') exibe o texto na tela da linha de comando e retorna o valor digitado até que seja pressionado enter. input('texto','s') é utilizado para entrada de strings.

Exemplo: id=input('Digite sua idade') guarda o número digitado em id; nom=input('Digite o seu nome', 's') guarda a string digitada em nom.

disp

Propósito: exibe texto na tela de comando.

Sintaxe: disp('texto').

keyboard

Propósito: invoca o teclado a partir de um arquivo MATLAB.

Sintaxe: keyboard.

pause

Propósito: gera pause na execução de um arquivo MATAB.

Sintaxe: pause; pause(n); pause on; pause off.

Descrição: pause interrompe a execução até que o usuário aperte alguma tecla. Pause(n) interrompe a execução por n segundos. Pause on (off) liga (desliga) o modo de pausa dos comandos subsequentes.

Exemplo: pause; pause(5); pause on; pause off.

Examine também os comandos: return, eval; feval; evalin; builtin; assignin; run; script functio; global; mfilename; lists exist; isglobal; nargchk; nargin; nargout; varargin; varargout; inputname; error; warning; lasterr; errortrap; fprintf; sprintf; uimenu; uicontrol; dbstop.

8.3 Funções de Controle de Baixo Nível

fopen

Propósito: abrir arquivo para leitura e/ou escrita.

Sintaxe: fopen('nome-do-arq','permissão')

Descrição: abre arquivo com o nome especificado para as operações descritas em permissão.

Exemplo: fopen('novo.m','r') abre o arquivo novo.m para leitura.

fclose

Propósito: fechar arquivo.

Sintaxe: fclose('nome-do-arq').

fopen

Propósito: abrir arquivo para leitura e/ou escrita.

Sintaxe: fopen('nome-do-arq','permissão')

Descrição: abre arquivo com o nome especificado para as operações descritas em permissão.

Exemplo: fopen('novo.m','r') abre o arquivo novo.m para leitura.

Examine também os comandos: fread; fwrite; fscanf; fprintf; fgetl; fgets; ferror; fseek; ftell; sprintf; sscanf.

8.4 Funções Especializadas: Matemáticas, Matriciais, Álgebra Linear

São funções de cálculos algébricos com matrizes e cálculos de álgebra linear.

Examine os comandos: cond; norm; rcond; rand; det; trace; null; orth; rref; chol; lu; qr; nnls; pinv; lscov; poly; hess; qz; rsf2csf; cdf2rddf; schur; balance; svd; expm; expm1; expm2; expm3; logm; log2m; sqrtm; funm; speye; sprandn; sprandsym; spdiags; sparse; full; spconvert; nnz; nonzeros; nzmax; spones; spalloc; issparse; spfun; spy; gplot; colmmd; symmmd; symrcm; colperm; tandperm; dmperm; normest; condest; sprank; symbfact; spparms; spaugment.; compan; hadamard; hankel; hilb; invhilb; pascal; rokker; toeplitz; vander; wilkinson.

8.5 Funções de Integração, Interpolação e Tratamento de Dados

São funções úteis para solução de equações diferenciais; interpolação de curvas e tratamento de pontos.

Examine os comandos: roots; poly; polyval; polyvalm; residue; polyfit; polyfit; polyder; conv; deconv; interp1; interp2; interpft; griddata; ode23; ode45; quad; quad8; fmin; fmins; fzero; fplot.

8.6 Funções de Controle de Cores, Imagens e Caracteres

São funções úteis para o controle de cores, manipulação de imagens, animação e controle de gráficos.

Examine os comandos: colormap; caxis; shading; hsv; gray; hot; cool; bone; copper; pink; flag; hsv2rgb; rgb2hsv; brighten; spinmap; rgbplot; surf; specular; diffuse; surfnorm; sound; saxis; auwrite; auread; mu2lin; lin2mu; string; abs; setstr; str2mat; strcomp; upper; lower; num2str; int2str; str2num; sprintf; sscanf; hex2num; hex2dec; dec2hex.

9 PROGRAMANDO EM ARQUIVOS .m

Podemos elaborar programas utilizando instruções assemelhadas aos comandos das linguagens procedurais, lançando dos arquivos .m do MATLAB.

Para isto basta criar um arquivo com extensão .m no diretório `/work` ou `/temp` do seu computador.

9.1 Exemplos de Aplicação de Programa '.m'

Exemplo 9.1: Programa `raizes.m`, que calcula as raízes de uma equação do 2º grau.

```
% Objetivo: Calcular raizes de uma equação do 2o.grau
% Prof. Baleeiro

disp(' ==== Entre com a, b e c ==== ');
a = input('      Coeficiente de x^2      ');
b = input('      Coeficiente de x        ');
c = input('      Termo independente de x  ');
if a == 0.
    disp(' ==== A equacao nao e do segundo grau ==== ');
    a = input('      Coeficiente de x^2 - a <> 0  ');
end

delta = b^2-4.*a*c;
delta

if delta == 0.
    ch = 0;
end
if delta < 0.
    ch = -1;
end
if delta > 0.
    ch = 1;
end
x(1) = (-b + sqrt(delta))/(2.*a);
x(2) = (-b - sqrt(delta))/(2.*a);
x
switch ch
    case 0, disp('      Raizes reais e iguais.')
    case -1, disp('      Raizes complexas conjugadas.')
    otherwise, disp('      Raizes reais e desiguais.')
end
% Fim
```

Para executar este programa é preciso apenas digitar no ambiente do MATLAB (isto é, no *prompt*) o nome do programa sem a extensão:

```
>>raizes < ENTER >
```

As raízes de qualquer polinômio podem ser encontradas de maneira mais direta no ambiente de trabalho do MATLAB. O comando utilizado é denominado "**roots**", que tem o formato a seguir:

```
>>roots([A B C]) < ENTER >
```

Onde A é coeficiente do termo x^2 , B é o coeficiente do termo x e C é o termo independente.

Por exemplo, as raízes da equação do 2º $x^2 - 5x + 6 = 0$ são:

```
» roots([1 -5 6])
```

```
ans =  
    3  
    2
```

Um outro exemplo de cálculo da raízes de um polinômio:

```
» roots([1 -6 11 -6])
```

```
ans =  
  
    3.0000  
    2.0000  
    1.0000
```

Ou seja, as raízes do polinômio $x^3 - 6x^2 + 11x - 6 = 0$ são: 1, 2 e 3.

Obs.: O comando **poly** faz a operação inversa do comando **roots**, ou seja, tendo-se as raízes, obtém-se o polinômio correspondente. Assim,

```
» poly([1 2 3])
```

```
ans =
```

```
1 -6 11 -6      ⇒      (polinômio  $x^3 - 6x^2 + 11x - 6 = 0$ )
```

Exemplo 9.2: Um outro exemplo de implementação de um código .m no MATLAB é mostrado a seguir em que a solução de um sistema algébrico linear é obtida pela aplicação do método iterativo de Jacobi. Denominaremos este programa de `jacobi.m`.

```
% Metodo de Jacobi para solucionar Ax = b
% Prof. Baleeiro
A = [6 -1 0;-1 4 1;0 1 6]
b = [-1; 1 ; 2]
[n,n] = size(A);
x = zeros(n,1);
r = zeros(n,1); % vetor de residuos
tol = 1e-2;

% Matriz de iteracao:
D = diag(diag(A))
L = tril(A)-D;
U = triu(A)-D;
E = -inv(D)*(L + U);
F = inv(D)*b;

% Autovalores de E
eig(E)
k = 0;
maior = inf;
while maior > tol
    x = E*x + F;
    r = A*x-b;
    maior = norm(r,inf) % norma infinita do vetor r
    pause
    k = k + 1;
end;
x
k
```


Para compreender o código mostrado anteriormente, segue uma breve descrição do método iterativo de Jacobi para solução de $Ax = b$. O método iterativo de Jacobi quando aplicado para resolver um sistema de equações lineares algébricas simultâneas, do tipo $Ax = b$ baseia-se numa partição da matriz dos coeficientes, da seguinte forma $A = L + U + D$, onde L e U são matrizes triangulares inferior e superior obtidas de A , respectivamente, excluindo-se os elementos da diagonal, e D é a matriz composta apenas dos elementos da diagonal da matriz A (observe que L , U e D não são matrizes resultantes da fatoração de A , trata-se apenas de uma partição).

Dessa forma, o sistema original pode ser reescrito como a seguir:

$$\begin{aligned} Ax &= b \\ Dx &= -(L+U)x + b \\ x &= Ex + F, \end{aligned}$$

de modo que,

$$E = -D^{-1}(L+U), F = D^{-1}b.$$

Designando E com a matriz de iteração, $E = -D^{-1}(L+U)$, dado $x^{(0)}$ e uma tolerância tol , o processo iterativo consiste em construir aproximações do vetor solução x do sistema algébrico, conforme a equação de iteração mostrada a seguir:

$$x^{(k+1)} = Ex^{(k)} + F, \text{ para } k = 0, 1, 2, 3, \dots$$

O processo iterativo deve continuar enquanto o critério de convergência não for alcançado. Uma forma que pode ser adequada para verificar a convergência é calcular a norma-infinita do vetor de resíduos, $r^{(k+1)} = Ax^{(k+1)} - b$, isto é, se $\|r^{(k+1)}\|_{\infty} \leq tol$, então a $x^{(k)}$ é a uma aproximação aceitável para solucionar $Ax = b$. Uma condição para convergência do método de Jacobi é que o maior módulo dos autovalores da matriz E seja inferior a 1.

Os passos do algoritmo do método de Jacobi para solução de sistemas $Ax = b$ são apresentados a seguir:

Dados A, b, tol e $x^{(0)}$

Calcule $D \leftarrow \text{diag}(\text{diag}(A)), L \leftarrow \text{tril}(A) - D, U \leftarrow \text{triu}(A) - D$

$$E = -D^{-1}(L + U)$$

$$F = D^{-1}b$$

Verifique se $\text{máximo}(|\lambda_i|) < 1$ (use o comando **eig(E)**)

Faça $k \leftarrow 0$, maior $\leftarrow \infty$

Enquanto maior $> tol$ faça

$$x^{(k+1)} \leftarrow Ex^{(k)} + F, \text{maior} \leftarrow \text{máximo}(|Ax^{(k+1)} - b|), k \leftarrow k + 1$$

Fim 'enquanto'

Saída x e k

Exemplo 9.3: Solução de sistema de equações algébricas lineares

$$\begin{bmatrix} -1 & 2 & 0 \\ 1/2 & 9 & 3 \\ 2 & 4 & 5 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ -2 \\ 7 \end{bmatrix}$$

A solução do exemplo 9.3 é facilmente obtida informando ao MATLAB, no próprio *prompt*, a matriz A e o vetor independente b . A solução x procurada para o sistema é obtida com o comando $\mathbf{x} = \mathbf{A} \backslash \mathbf{b}$, como a seguir.

```

» A = [-1, 2, 0; 1/2, 9, 3; 2, 4, 5]
A =
   -1.0000    2.0000    0.0000
    0.5000    9.0000    3.0000
    2.0000    4.0000    5.0000

» b = [3; -2; 7]
b =
     3
    -2
     7

» x = A\b

x =
   -6.1923
   -1.5962
    5.1538

```

10 MATEMÁTICA SIMBÓLICA

Exemplo 10.1: Solução de integral definida

$$\int_0^1 \frac{1}{x^2+1} dx$$

O exemplo 10.1, para ser resolvido, requer que seja informado ao MATLAB que há uma variável x , que deve ser tratada como um dado simbólico. Isto é feito através do comando:

```
» syms x
```

Os comandos seguintes mostram a finalização da solução do exemplo 10.1:

```
» int(1/(x^2 + 1), 0, 1)

ans =

1/4*pi
```

Realmente, o valor resultante da integração da função $f(x) = \frac{1}{x^2+1}$ entre 0 e 1 é de fato $\frac{\pi}{4}$.

Exemplo 10.2: Sistema massa-mola. Este sistema será, também, analisado usando o Simulink, na seção 11 deste texto.

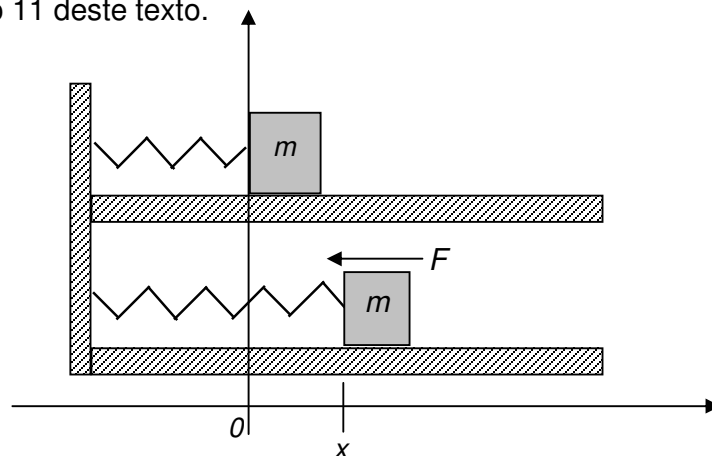


Fig. 10.1: Sistema massa-mola.

As equações do sistema massa-mola são dadas na seção 11 e, assim, passa-se diretamente ao programa `.m` desenvolvido e os resultados mais importantes.

```
% Sistema massa-mola
% Prof. José Wilson

k =input('constante de elasticidade da mola (N/m)Ex.:700= ');
xmax=input('deslocamento máximo da mola xmax (cm);Ex.: 20= ');
m =input('massa do bloco ligado à mola (g); Ex.:500= ');

xmax = xmax/100;
m = m/1000;

x = [-xmax:0.001:xmax];

Ettotal = (1/2)*k*(xmax^2); % Energia total

F = -k*x; % Força na mola

Ep = (1/2)*k*(x.^2); % Energia potencial

Ec = Ettotal - Ep; % Energia cinética - Conservação de energia

v = sqrt(2*Ec./m); % Velocidade da massa-Conservação de energia
vmax = sqrt(2*Ettotal/m); % Velocidade máxima

figure(1)
plot(x*100,Ep,'r')
hold on
plot(x*100,Ec,'k')
plot(x*100,Ettotal,'m')
axis([-xmax*100 xmax*100 0 Ettotal])
xlabel('deslocamento (cm)')
ylabel('energia cinética, potencial e total (J)')
grid on
hold off

figure(2)
plot(x*100,v)
axis([-xmax*100 xmax*100 0 vmax])
xlabel('deslocamento (cm)')
ylabel('velocidade da massa (m/s)')
grid on
```

Selecionou-se para este exemplo os gráficos de energia *versus* deslocamento e velocidade *versus* deslocamento, que serão comparados com os resultados equivalentes, quando da utilização do Simulink. Observar que, no presente resultado, a velocidade mostrada é apenas no sentido do deslocamento negativo para o positivo. A velocidade no sentido inverso (que é mostrada no modelo do Simulink) é negativa.

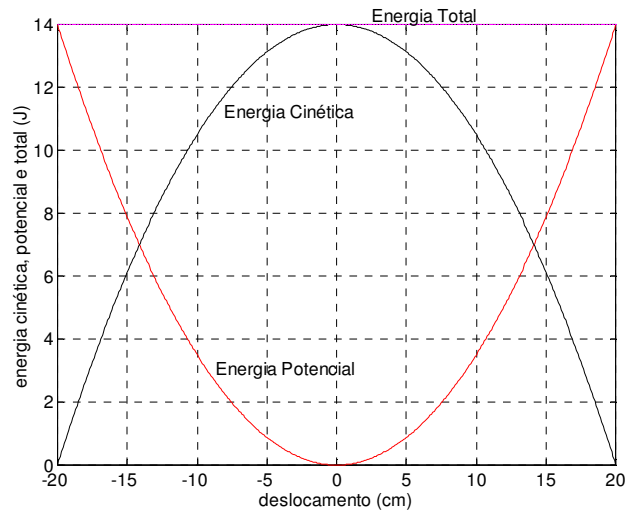


Fig. 10.2: Energia Cinética, Potencial e Total *versus* Deslocamento

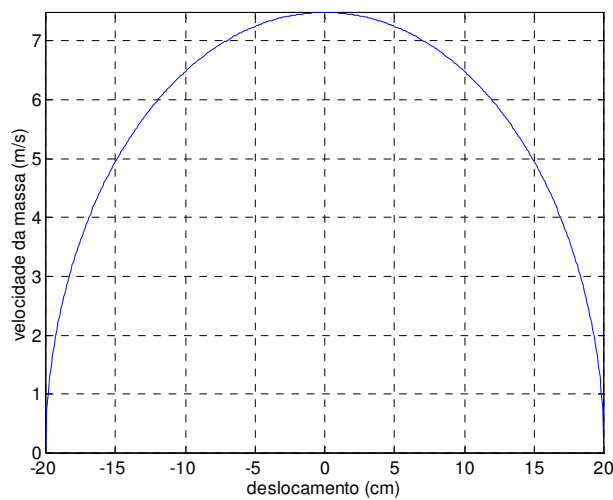


Fig. 10.3: Velocidade *versus* Deslocamento

Exemplo 10.3: Lançamento de projétil. Neste exemplo entra-se com os dados de altura inicial, velocidade inicial, posição horizontal inicial e ângulo de lançamento. O programa calcula o tempo transcorrido até o projétil atingir o solo, a velocidade no momento em que toca o solo e a altura máxima atingida.

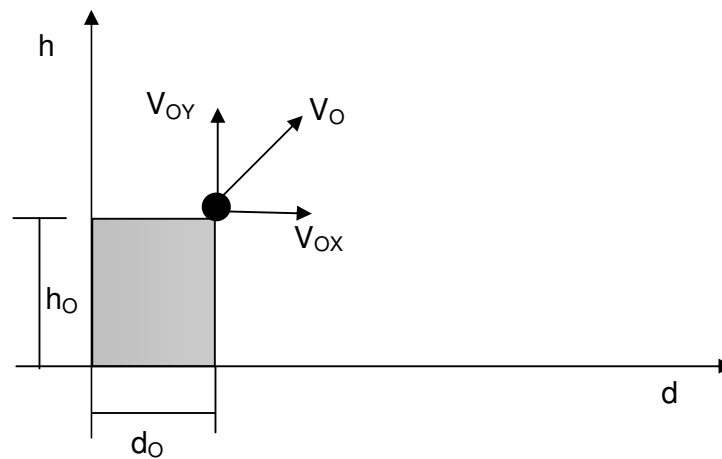


Fig. 10.4: Lançamento de projétil

```
% Lançamento de projétil
% Prof. José Wilson

ho = input('Altura inicial ho (m)          Ex.: 40      = ');
vo = input('velocidade inicial vo (m/s)    Ex.: 25      = ');
alfa = input('ângulo de lançamento alfa (grau) Ex.: 45  = ');
do = input('posição inicial do projétil (m) Ex.: 0      = ');

alfa = alfa*pi/180;
vx0 = vo*cos(alfa);
vy0 = vo*sin(alfa);
g = -9.8;           % Aceleração da gravidade

h = [0.5*g vy0 ho]; % Para cálculo do tempo máximo

tlimites = roots(h); % Cálculo do tempo limite - tem 2 valores

if tlimites(1) > 0
    tmax = tlimites(1);
else
    tmax = tlimites(2);
end

t = [0:0.01:tmax];

vx = vx0;
vy = vy0 + g*t;

thmax = abs(vy0/g); % tempo para atingir altura maxima

d = do + vx0*t;
h = ho + vy0*t + 0.5*g.*t.^2;
```

```

hmax = ho + vyo*thmax + 0.5*g*thmax^2; % altura maxima
vmax = vyo + g*thmax; % velocidade maxima
dmax = do + vxo*thmax; % distancia maxima alcancada

figure(1)
plot(d,h)
axis([0 dmax 0 hmax])
xlabel('distância horizontal (m)')
ylabel('altura (m)')
grid on

figure(2)
plot(t,h)
axis([0 tmax 0 hmax])
xlabel('tempo (s)')
ylabel('altura (m)')
grid on

figure(3)
plot(t,d)
axis([0 tmax 0 dmax])
xlabel('tempo (s)')
ylabel('distância horizontal (m)')
grid on

```

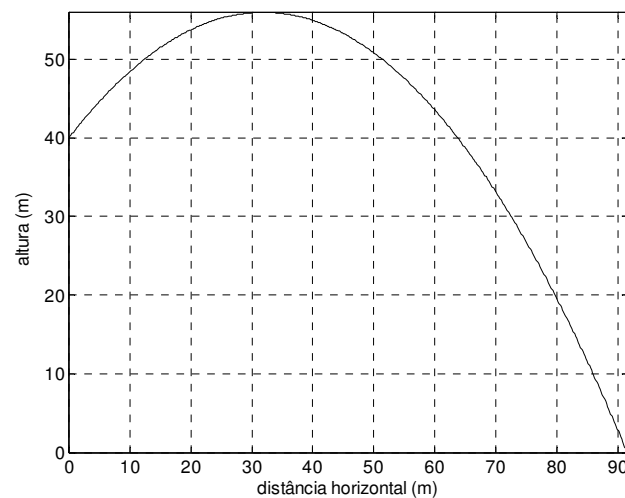


Fig. 10.5: Altura do projétil *versus* alcance horizontal

A altura máxima, a distância máxima e o tempo decorrido até atingir o solo são:

$$h_{max} = 55,9 \text{ m} \quad d_{max} = 91,6 \text{ m} \quad t_{max} = 5,18 \text{ s}$$

10.1 Uso da Matemática Simbólica

O MATLAB possui um Toolbox que permite ao usuário operar com matemática simbólica. Sabemos que variáveis numéricas são usadas para armazenar valores numéricos (e.g., $x = 2$) e vetores de caracteres são usados para armazenar textos (e.g., $s = 'Jonas'$). O Toolbox de Matemática Simbólica usa objetos simbólicos para representar variáveis e operações (e.g., $x = \text{sym}('x')$). Neste tipo de aplicação não é necessário que as variáveis tenham um valor numérico predefinido.

A matemática simbólica consiste da resolução de equações simbólicas (isto é, na forma literal) e da execução de operações de cálculo diferencial/integral e álgebra por meio da aplicação de regras conhecidas e identidades a determinados símbolos, exatamente como você aprendeu a resolvê-las nos cursos de cálculo e álgebra.

Exemplo 10.4: Obter, de forma literal, a derivada em relação a x da função $f(x)$ dada a seguir:

$$f(x) = ax^2 + bx$$

É necessário, em primeiro lugar, informar ao MATLAB que x , a , b e f não são objetos simbólicos, ou seja, não possuem valores numéricos. Então:

```
» syms x a b f
```

Em seguida, define-se a função e utiliza-se o comando **diff** para concluir a operação simbólica de derivação.

```
» f = a*x^2 + b*x

f =
a*x^2+b*x

» diff(f,x)

ans =
2*a*x+b
```


Exemplo 10.5: Outro exemplo (mais complexo) do uso da matemática simbólica do MATLAB. Cálculo da área interna de uma elipse dada no sistema de coordenadas xy pela equação:

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$$

Deseja-se obter uma fórmula para calcular sua área interna A dados a e b . A integral mostrada a seguir é usada no cálculo dessa área.

$$A = 4 \frac{b}{a} \int_0^a \sqrt{a^2 - x^2} dx$$

A solução deste problema é conhecida dos estudantes que fizeram cursos de cálculo diferencial e integral. O resultado é $A = \pi ab$. Utilize a *Toolbox* de Matemática Simbólica para comprovar este resultado. No caso, você deverá lançar mão do comando `int`, que calcula integral de uma função dada em intervalo definido.

10.2 Gráficos em Duas Dimensões

Embora existam várias formas para se fazer gráficos, nesta seção vamos nos ater apenas à exemplificação do emprego do comando `plot`. Esse comando já foi utilizado em alguns exemplos anteriores.

Exemplo 10.6: Gráfico em duas dimensões

```
» x = linspace(-2,2,30);
» y = exp(-x.^2);
» plot(x,y)
```

Esse exemplo cria 30 pontos no intervalo $-2 \leq x \leq 2$ para formar o eixo horizontal do gráfico e cria o vetor y contendo a exponencial e^{-x^2} dos pontos em x . O

comando `plot` abre uma janela gráfica, ajusta os eixos para acomodar os dados, marca os pontos e conecta-os a linhas retas. O resultado é ilustrado na Fig. 10.6.

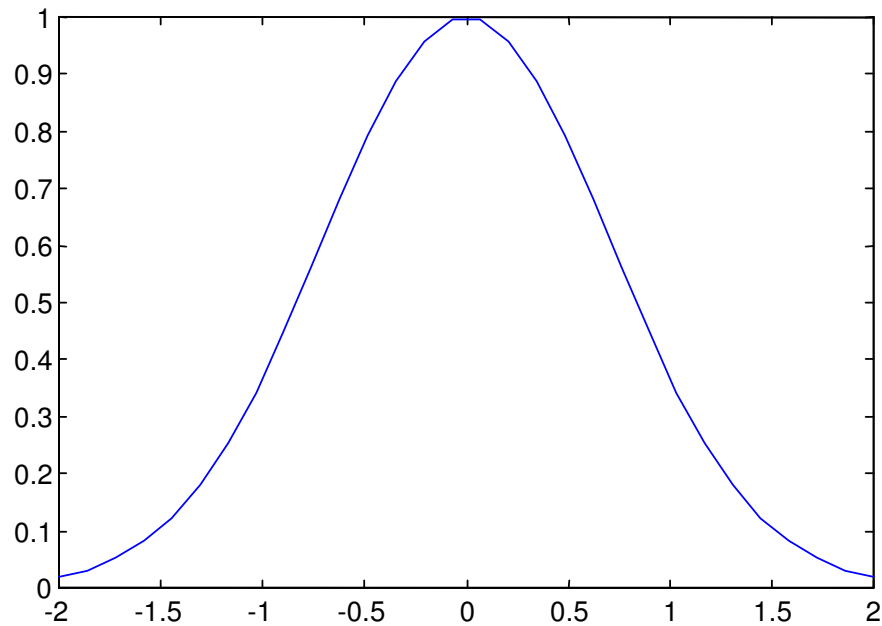


Fig. 10.6: Saída gráfica obtida com os comandos indicados anteriormente.

10.3 Gráficos em Três Dimensões

Para elaborar gráficos tridimensionais o MATLAB possui diversos comandos poderosos. Sugerimos que você tente as seqüências de comandos propostas nos exemplos que seguem.

Exemplo 10.7: Use do comando `plot3` para desenhar uma hélice, de acordo com a seqüência:

```
» t = linspace(0, 10*pi);  
» plot3(sin(t), cos(t), t)
```

Exemplo 10.8: Uso do comando `mesh` para gráficos tridimensionais. Use a sequência de comandos para desenhar uma superfície.

```
» x = -7.5:.5:7.5;  
» y = x;  
» [X,Y] = meshgrid(x,y);  
» R = sqrt(X.^2 + Y.^2) + eps;  
» Z = sin(R) ./ R;  
» mesh(X,Y,Z)
```

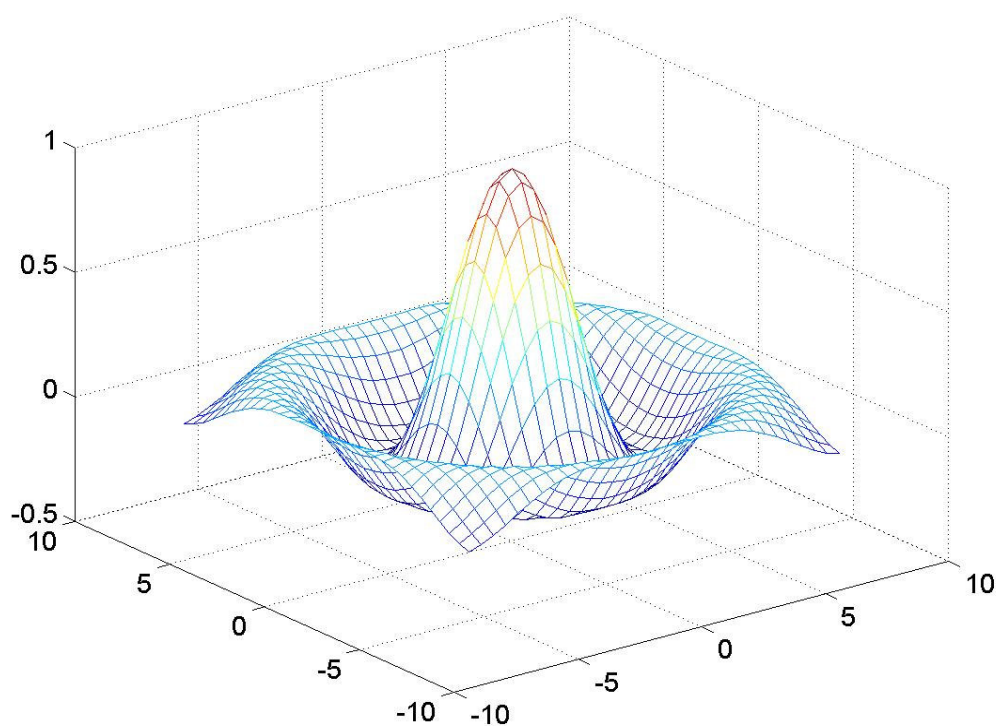


Fig. 10.7: Saída gráfica obtida com a sequência de comandos do exercício 3.

11 SIMULINK

O SIMULINK é um Programa de Simulação dinâmica de sistemas a partir de seu modelo no espaço de estados, do seu modelo no domínio da frequência ou do seu modelo no domínio do tempo.

O simulink é iniciado digitando-se **simulink** na linha de comando do MATLAB, ou ainda selecionando-se o botão de início do SIMULINK. Uma vez iniciado é apresentada a tela da figura a seguir:

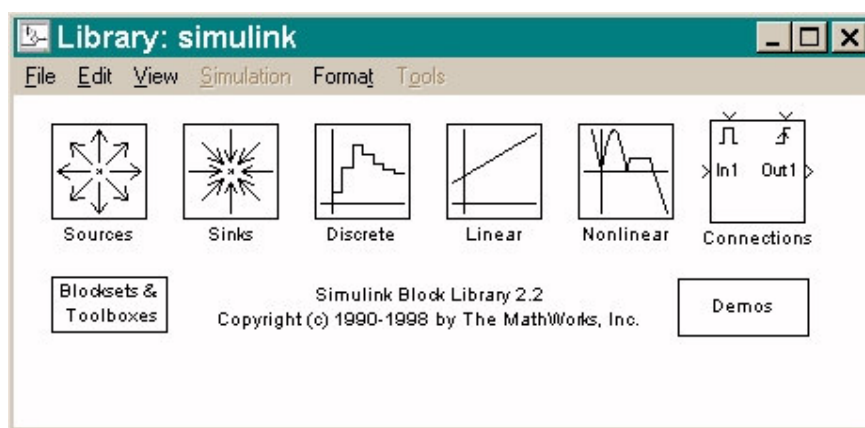


Fig. 11.1: Tela Inicial do Simulink

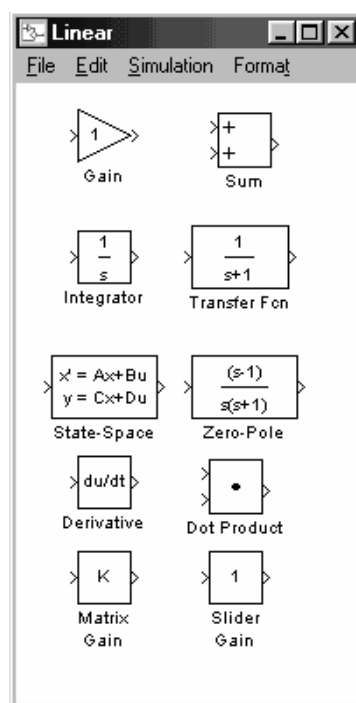


Fig. 11.2: Funções disponíveis para a classe linear do Simulink.

Os diversos blocos de simulação podem ser arrastados para o arquivo, com a finalidade de montar o modelo para simulação. Cada classe de funções possui uma janela própria (caixa), veja por exemplo os blocos disponíveis para as funções da classe linear na figura anterior.

A seguir são dados alguns exemplos de emprego do Simulink em problemas de Matemática, Física, Circuitos Elétricos, Controle de Processos e Outros.

Exemplo 11.1: Equação Diferencial de Primeira Ordem

Considere $u = u(t)$, $u \in \mathbf{R}$, $t \in \mathbf{R}$. Seja a equação diferencial de primeira ordem em $u(t)$, com valor inicial conhecido, $u(0) = 1$.

$$\frac{du}{dt} + 2tu = 0 \quad (11.1)$$

Analiticamente é fácil mostrar que uma solução dessa equação diferencial é:

$$u(t) = u(0)e^{-t^2} \quad \therefore \quad u(t) = e^{-t^2} \quad (11.2)$$

Vamos resolver a equação diferencial (11.1) utilizando o Simulink e, ao final, obteremos a resposta na forma gráfica.

Solução:

PASSO 1: Inicialmente escreve-se a equação (11.1) de uma forma mais adequada para diagrama de blocos. Transpondo os termos, chega-se à forma indicada em (11.3):

$$u = -2 \int t u dt \quad (11.3)$$

Ao fator -2 chamamos de ganho; o integrando é resultado de um produto de duas funções, t e u . Portanto, a resposta u é a resultante da saída de um integrador multiplicado pelo ganho -2 .

PASSO 2: A partir da janela mostrada na Figura 11.1, selecionamos os blocos necessários para expressar a relação (11.3).

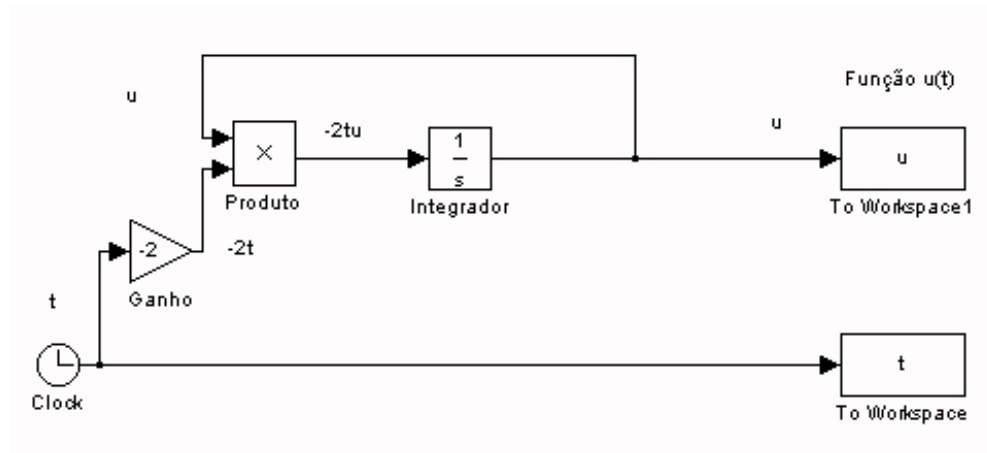


Fig. 11.3: Diagrama de blocos para resolver o Exemplo 11.1

A condição inicial é informada ao bloco integrador. O eixo dos tempos é representado pelos blocos 'Clock' e 'To Workspace'. Em ambos os blocos 'To Workspace' é preciso definir o número de pontos e o tamanho do passo numérico (neste exemplo, tomamos 5000 e 0.01, respectivamente).

PASSO 3: Para executar o arquivo criado, definido pelo diagrama de blocos, nos ícones da mesma janela, pressionamos *Simulation* e, em seguida, *Start*.

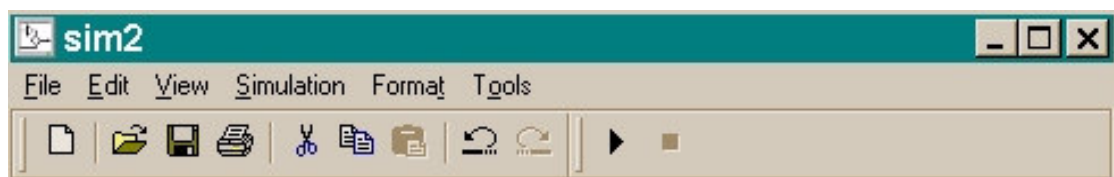


Fig. 11.4: Para executar o arquivo de blocos.

PASSO 4: Para visualizar o resultado, no *prompt* do MATLAB, digitamos o comando:

```
» plot(t,u)
```

A saída gráfica da resposta $u(t) \times t$ é como ilustra a Fig. 11.5.

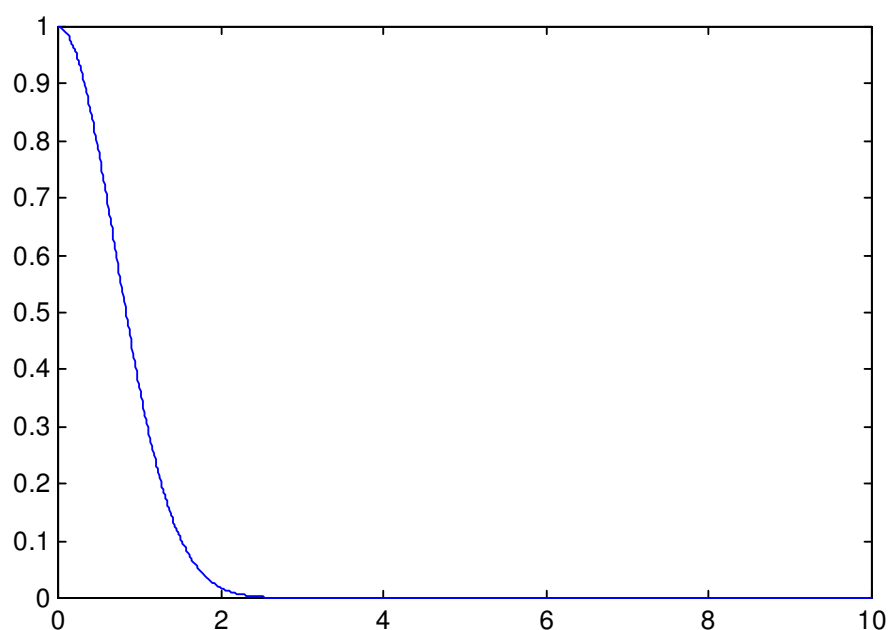


Fig. 11.5: Gráfico de $u(t) \times t$ para $t \geq 0$.

Finalizamos, desse modo, a solução da equação diferencial (11.1), que resultou na representação gráfica da função e^{-t^2} para $t \geq 0$.

Exemplo 11.2: Sistema Massa-Mola

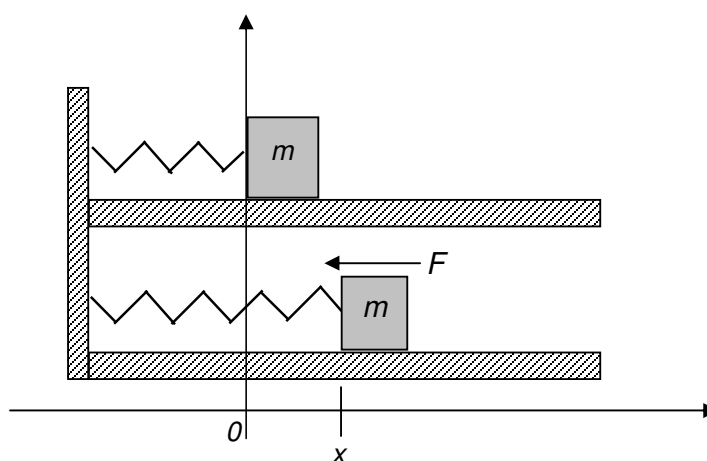


Fig. 11.6: Representação do Sistema Massa-Mola

Equações básicas da Física:

$$\text{Aceleração: } a = \frac{dv}{dt} = \frac{d^2x}{dt^2} \quad (11.4)$$

$$\text{Velocidade: } v = \frac{dx}{dt} = \int a dt \quad (11.5)$$

$$\text{Deslocamento: } x = \int v dt \quad (11.6)$$

Equações básicas do Sistema massa-mola:

$$\text{Força: } F = -kx = ma \quad \therefore \quad kx = m \frac{d^2x}{dt^2} \quad \therefore \quad a = \frac{d^2x}{dt^2} = -\frac{k}{m}x \quad (11.7)$$

$$\text{Energia potencial: } E_p = \frac{1}{2}kx^2 \quad (11.8)$$

$$\text{Energia cinética: } E_c = \frac{1}{2}mv^2 \quad (11.9)$$

Verifica-se das equações que a aceleração, no sistema massa-mola, é diretamente proporcional ao deslocamento 'x'. O fator de proporcionalidade é a constante '-k/m'. Essa é a informação inicial para começar o modelo dado na Fig. 11.7.

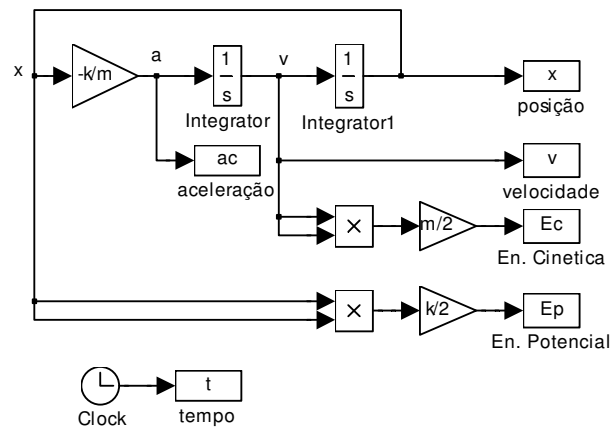


Fig. 11.7: Modelo do Simulink para o Sistema Massa-Mola

Para a simulação do sistema é necessário fornecer um valor inicial para um dos dois blocos de integração. Essa informação será, no caso, o limite para a variável de saída. Por exemplo, desejando-se limitar o deslocamento 'x' entre os valores -20 cm (-0.2 m) e 20 cm, fixa-se em 0.2 o valor inicial da segunda integral. Uma outra

informação fundamental é o valor da constante de proporcionalidade 'k' e o valor da massa 'm'. Esses valores podem ser digitados diretamente no *prompt* (área de trabalho do MATLAB), como mostrado a seguir:

```
» k = 700;  
» m = 0.5;
```

As figuras que seguem são resultantes do modelo da Fig. 11.7.

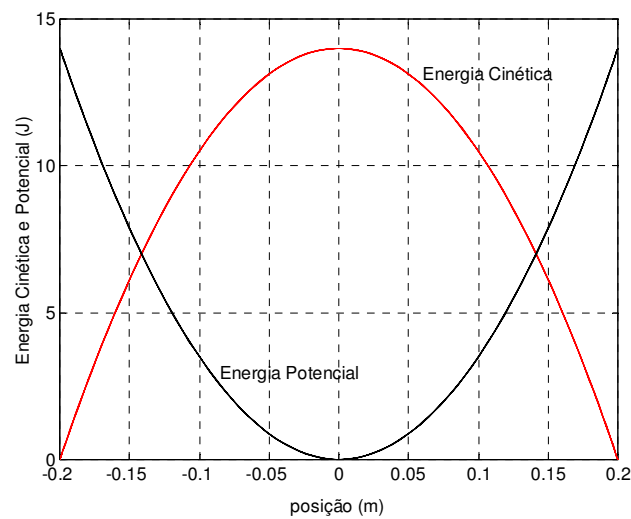


Fig. 11.8: Energia cinética e potencial *versus* deslocamento

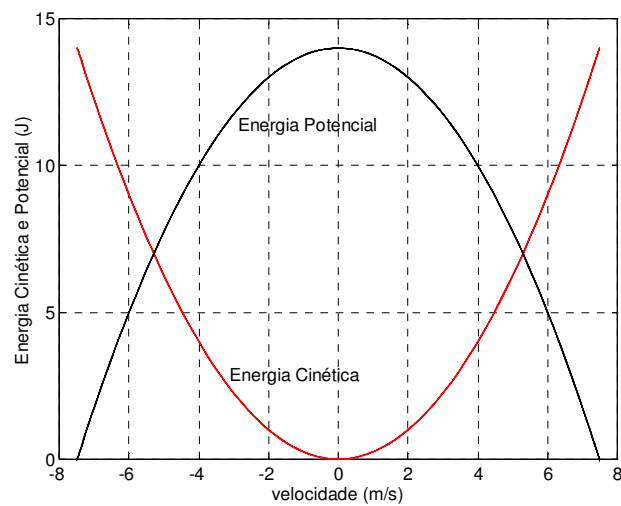


Fig. 11.9: Energia cinética e potencial *versus* velocidade

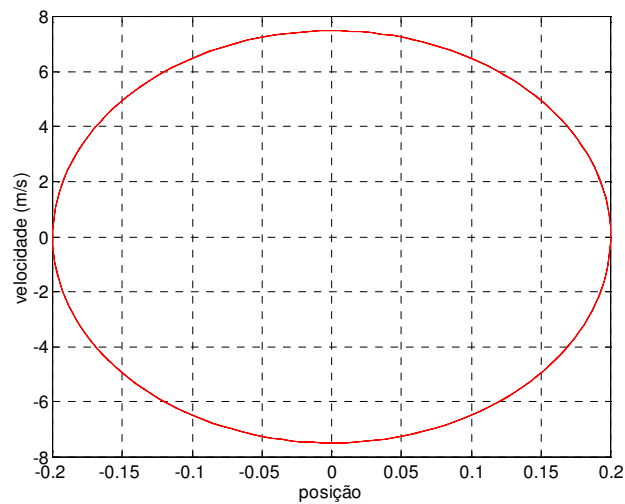


Fig. 11.10: Velocidade *versus* deslocamento

Exemplo 11.3: Circuito RC Série

Considere o circuito elétrico da Fig. 11.11, que possui um resistor e um capacitor em série alimentados por uma fonte constante. O capacitor possui uma tensão inicial $v(0) = 10\text{ V}$ e deseja-se obter a resposta $v(t) \times t$ para $t \geq 0$.

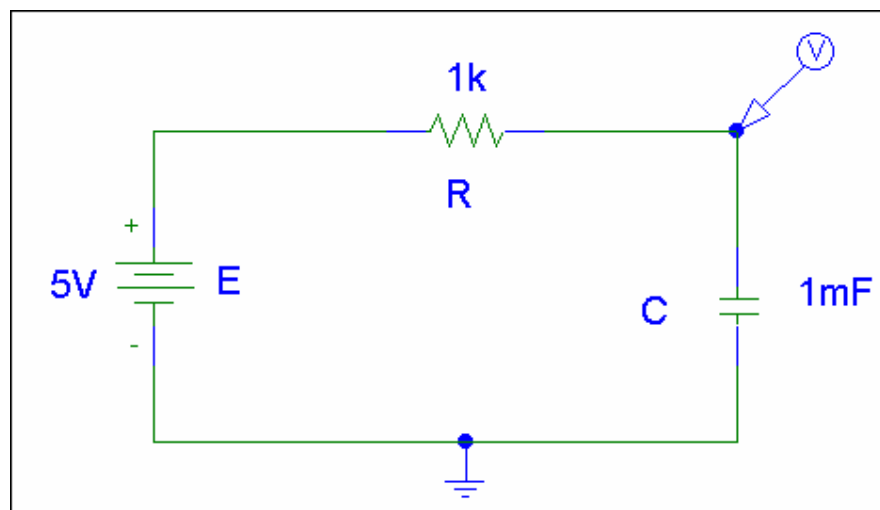


Fig. 11.11: Circuito elétrico RC série com uma fonte de tensão contínua.

Modelagem Matemática do Circuito

A modelagem matemática do circuito é obtida aplicando-se a 2ª Lei de Kirchhoff ao percurso fechado, e usando a forma genérica e , para expressar tensão:

$$e - v_R - v = 0$$

Por outro lado, sabemos relacionar a tensão no resistor e a tensão no capacitor com a corrente que os atravessam, $i(t)$:

$$v_R = Ri(t)$$

$$i(t) = C \frac{dv}{dt} \quad \therefore \quad e - Ri(t) - v = 0 \quad \text{ou} \quad e - RC \frac{dv}{dt} - v = 0$$

Assim, a equação diferencial geral fica:

$$\frac{dv}{dt} + \frac{1}{RC}v = \frac{1}{RC}e$$

Substituindo os valores de $R = 1\text{k}\Omega$ e $C = 1\text{mF}$ e $e = E = 5\text{V}$ na equação anteriormente mostrada, resulta na equação mostrada a seguir:

$$\frac{dv}{dt} + v = 5$$

$$\text{onde } v(0) = 10.$$

Solução do Circuito Utilizando o Simulink

Para utilizar o Simulink devemos expressar a equação diferencial da seguinte maneira:

$$v = \frac{1}{RC} \int (e - v) dt$$

A expressão acima é conseguida facilmente apenas isolando o termo $\frac{dv}{dt}$ e depois aplicando a integração (que é a operação inversa da derivação). Essa forma é

ideal para a simulação usando o Simulink. Constrói-se então o modelo mostrado na Fig. 11.12.

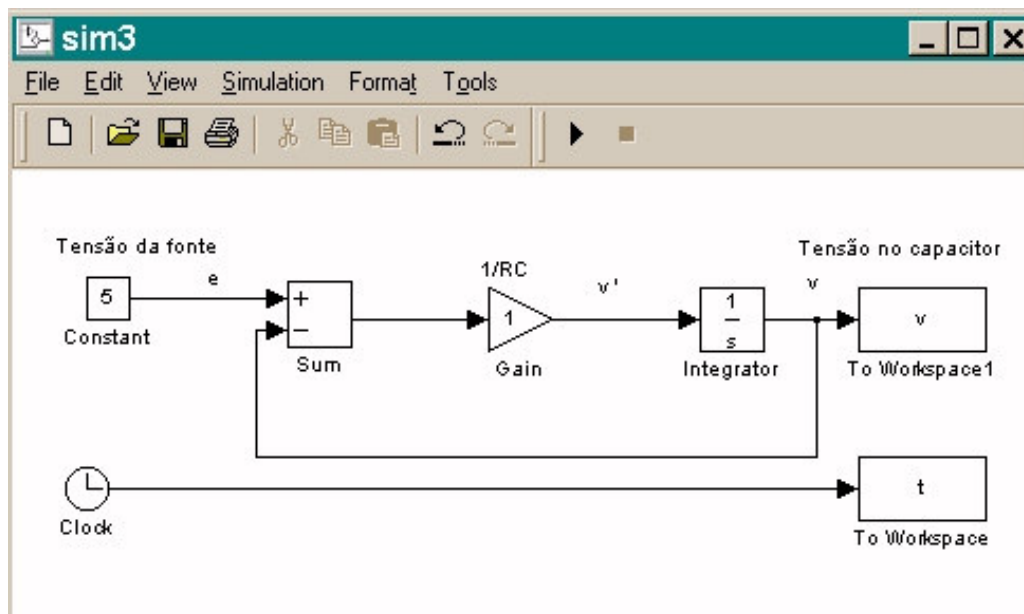


Fig. 11.12: Diagrama de blocos do Simulink para o circuito elétrico da Fig. 11.11

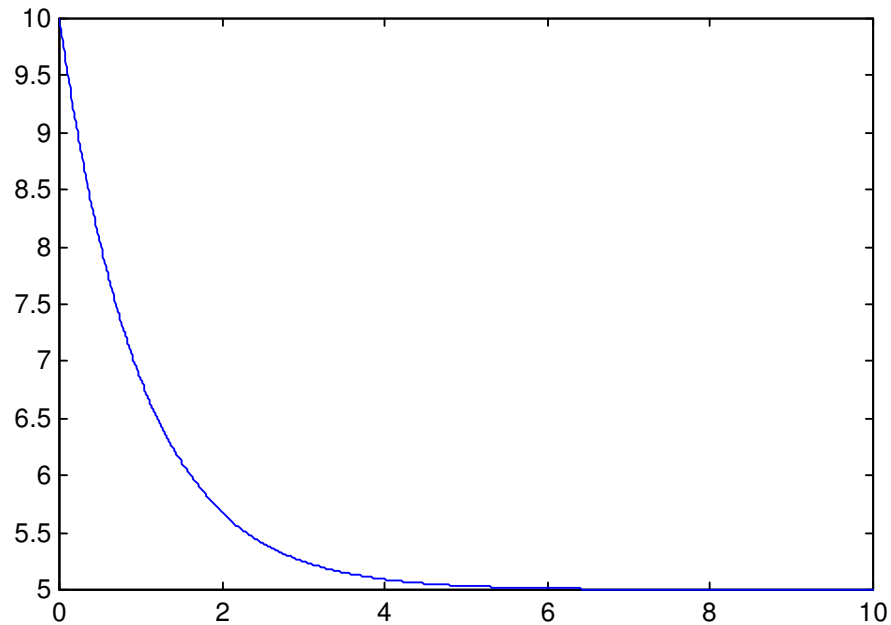


Fig. 11.13: Saída gráfica $v(t) \times t$ para $t \geq 0$ para o circuito elétrico da Fig. 11.11.

12 EXERCÍCIOS RESOLVIDOS

01) Considere a seguinte equação diferencial: $\frac{d^2y}{dt^2} + 12\frac{dy}{dt} + 32y = 32u(t)$. Obter a solução $y(t)$ supondo todas as condições iniciais iguais a zero, usando a transformada Laplace.

A transformada Laplace é:

$$s^2Y(s) + 12sY(s) + 32Y(s) = \frac{32}{s}$$

A solução para $Y(s)$ é:

$$Y(s) = \frac{32}{s(s^2 + 12s + 32)} = \frac{32}{s^3 + 12s^2 + 32s}$$

Resolver a equação diferencial utilizando a função **residue** do MATLAB.

Solução:

```
>> numy = 32
>> deny = [1 12 32 0]
>> [r p k] = residue(numy, deny)
```

O resultado obtido é o seguinte:

r =

1
-2
1

p =

-8
-4
0

k =

[]

que se traduz em:

$$Y(s) = \frac{1}{s} - \frac{2}{(s+4)} + \frac{1}{(s+8)}$$

$$y(t) = (1 - 2e^{-4t} + e^{-8t}).$$

02) O movimento da massa, designado por $y(t)$, é descrito pela equação diferencial:

$$M \frac{d^2 y(t)}{dt^2} + b \frac{dy(t)}{dt} + ky(t) = r(t)$$

onde:

M = massa em kg,

b = força de atrito (amortecimento viscoso),

k = constante de mola de uma mola ideal,

r = força em Newton.

A resposta dinâmica livre, $y(t)$, do sistema

$$y(t) = \frac{y(0)}{\sqrt{1-\zeta^2}} e^{-\zeta \omega_n t} \sin(\omega_n \sqrt{1-\zeta^2} t + \theta)$$

onde:

$$\zeta = \text{relação de amortecimento} = \frac{b}{2\sqrt{kM}}$$

$$\omega_n = \text{freqüência natural do sistema} = \sqrt{\frac{k}{M}} \quad \text{e} \quad \theta = \cos^{-1} \zeta.$$

Fazendo-se $y(0) = 0.15$ m, $M = 1$ kg, $k = 2$, para $b = 1$, simule no MATLAB para obter o gráfico referente a resposta natural do sistema e verifique que a resposta é subamortecida.

Solução:

```
>> y0 = 0.15;
>> wn = sqrt(2);
>> quisi = 1/(2*sqrt(2*1));
>> c1 = (y0/sqrt(1-quisi^2));
>> teta = acos(quisi);
>> t=[0:0.1:10];
>> c2 = exp(-quisi*wn*t);
>> c3 = sin(wn*sqrt(1-quisi^2)*t+teta);
>> y=c1*c2.*c3;
>> plot(t,y)
```

O gráfico da resposta natural do sistema obtido com o MATLAB para os parâmetros dados é mostrado na Fig. 12.1.

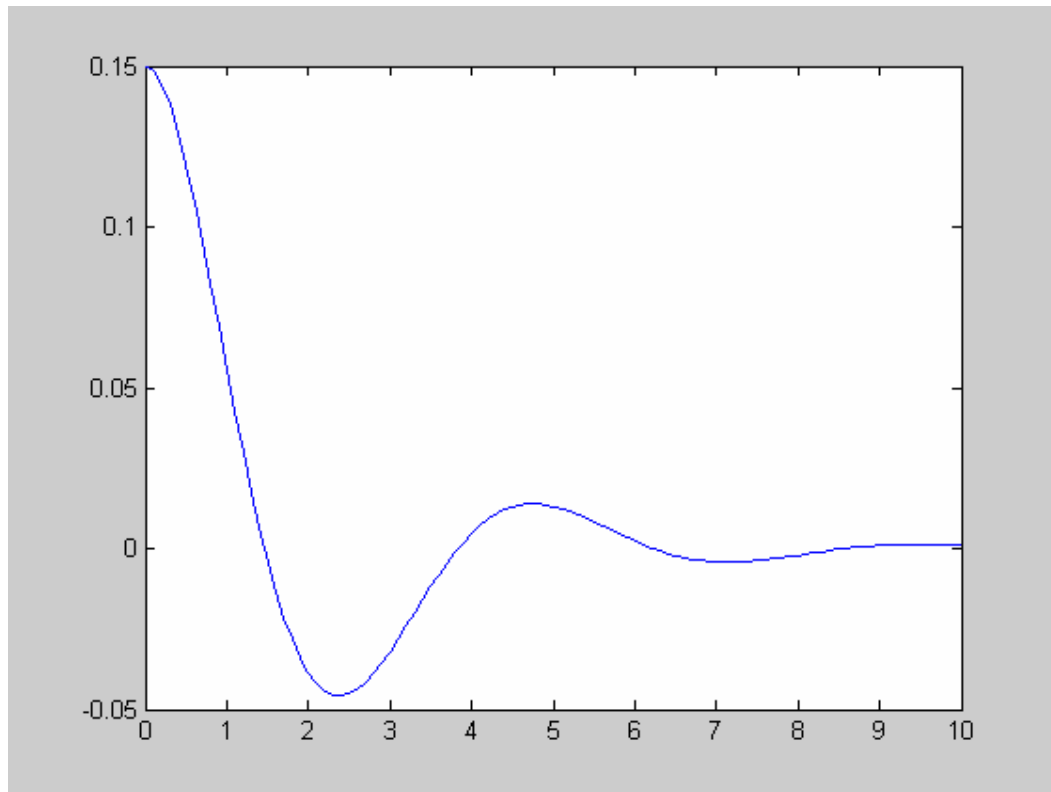


Fig. 12.1: Saída gráfica $y(t) \times t$ para $t \geq 0$.

03) Dada a função de transferência, obtenha o que se pede:

- a) calcular zeros e pólos;
- b) exibir a função de transferência;
- c) obter o mapa dos zeros e pólos.

$$G(s) = \frac{10s + 70}{s^2 + 30s + 200}$$

Solução:

```
>> numg = [10 70];
>> zeros = roots(numg)
>> deng = [1 30 200]
>> polos = roots(deng)
>> tfout(numg,deng)
>> sys = zp(zeros,polos,1);
>> pzmap(sys)
```

04) Um sistema dinâmico linear é representado no espaço de estados pelas seguintes equações matriciais:

$$\dot{x} = Ax + Bu$$

$$y = Cx + Du$$

Obtenha a representação no espaço de estados sob a forma de variáveis de fase das seguintes funções de transferência usando a função **tf2ss** do MATLAB:

$$T_1(s) = \frac{2s + 1}{s^2 + 7s + 9}$$

Solução:

```
>> num=[2 1]
>> den=[1 7 9]
>> [A,B,C,D]=tf2ss(num,den)
```

As matrizes resultantes são:

A =

$$\begin{bmatrix} -7 & -9 \\ 1 & 0 \end{bmatrix}$$

B =

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

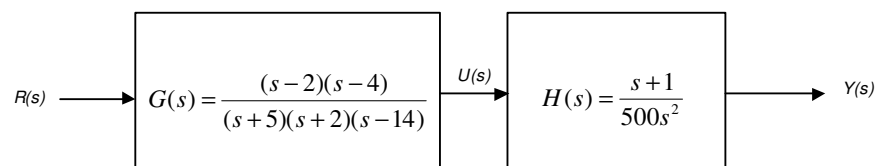
C =

$$\begin{bmatrix} 2 & 1 \end{bmatrix}$$

D =

$$\begin{bmatrix} 0 \end{bmatrix}$$

05) Sejam as seguintes funções de transferência nos blocos em cascata:



Use a função **series** do MATLAB para transformá-las em uma só.

Solução;

```
>> n1=[1 -2]
>> n2=[1 -4]
>> ng=conv(n1,n2)
>> d1=[1 5]
>> d2=[1 2]
>> d3=[1 -14]
>> dg=conv(d1,conv(d2,d3))
>> nh=[1 1]
>> dh=[500 0 0]
>> [n,d]=series(ng,dg,nh,dh)
```

A função de transferência resultante dos dois blocos em cascata é a seguinte:

$$\frac{s^3 - 5s^2 + 2s + 8}{500s^5 - 3500s^4 - 44000s^3 - 70000s^2}.$$

06) Analise a seqüência de comandos do MATLAB apresentada a seguir.

a) extraindo e inserindo

```
A = [ 1 3 4; -1 3 0; 6 -1 2]
C = [1 + i, 1 - i; 2i, 1 - i]
b = [1; 3; -1]
A(:,2)
A(1,:)
A(2,3)
M = [A,b]
M(end)
transpose(C)           % diferente de C' se C for complexa
C'
A(3:6)
A(:,2) = [0; 0; 0]     % insere o vetor da direita no lugar da
coluna 2
```

b) criando matrizes

```
>> D = ones(3)
>> Z = eye(3)
>> R = rand(3)
```

c) operações sobre matrizes e sobre elementos de matrizes

```
>> trace(A)          % soma algébrica dos elementos da diagonal de A
>> [m,n] = size(A)
>> [L,U,P] = lu(A)    % obtém a fatoração PLU de A
>> chol(A)            % somente para matriz definida positiva
```

d) decomposição espectral

```
Y = [4+i, -1+i, -3-2i;-1+i, 3-i, -2+i;-3-2i, -2+i, 5+i]
```

```
[P,D] = eig(Y)
```

```
W = P*D*P^-1
```

```
Y-W
```

```
Z = inv(Y)
```

```
disp(' o produto tem que resultar na matriz identidade');
```

```
Z*Y
```

```
H = P^-1;
```

```
B = (D^-1)*H;
```

```
U = P*B
```

```
Z-U
```

07) Obtenha uma raiz da função unidimensional $f(x) = e^{-x} - x + 1$ com estimativa inicial $x^{(0)} = 0.8$.

Solução:

```
>> fzero('exp(-x)-x+1',0.8) % obtem um zero da funcao
```

08) Dada a função de transferência

$$T(s) = \frac{C(s)}{R(s)} = \frac{1}{s+1},$$

determine a resposta do sistema ao degrau unitário, $R(s) = \frac{1}{s}$.

Solução:

```
>> clf
>> numg = [0 1];
>> deng = [1 1];
>> G = tf(numg,deng)
>> step(G)
>> title(' Amplitude versus tempo ')
pause
```

A resposta é como mostrada na Fig.12.2.

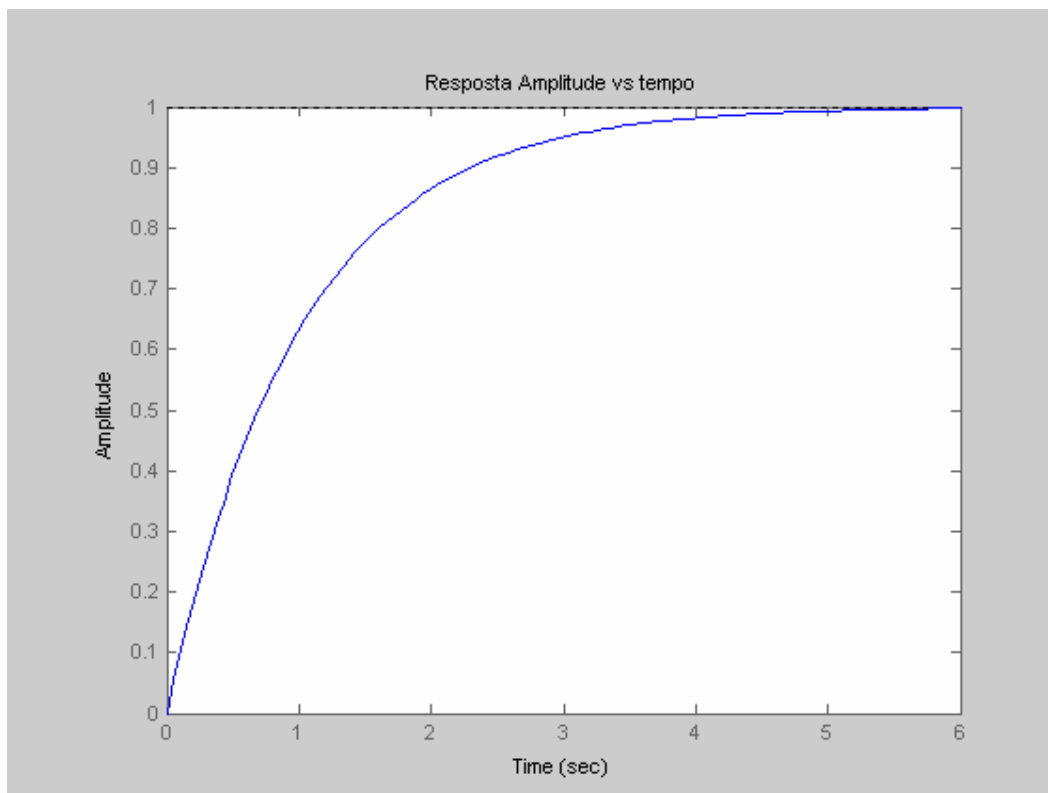


Fig. 12.2: Saída gráfica $c(t)$ com entrada $r(t) = u(t)$, para $t \geq 0$.

09) Obtenha 5 termos do desenvolvimento em Série de Taylor da função $f(x) = \cos x$.

Solução:

```
>> syms x
>> taylor(cos(x),5)
```

ans =

```
1-1/2*x^2+1/24*x^4
```

10) Ao final de sua seção de trabalho no *prompt* salve-a com o comando **save** em um arquivo com extensão .mat.

Solução:

```
>> save hoje.mat
```

Dessa forma, o comando **save** salva o *workspace* em um arquivo .mat.

Em seguida, utilize o comando **clear all** para limpar o conteúdo da memória. Através do comando **load hoje.mat** é possível carregar o conteúdo da memória permitindo a retomada do trabalho que estava sendo realizado.

11) Elabore um programa em arquivo .m que utilize a **function** do MATLAB.

Solução:

```
function [f] = fun2(a,b,x)    % em arquivo .m separado
global f;
    f = (b/a)*sqrt(a^2 - x.^2);
return;
```

```
% o conteudo a seguir em arquivo .m separado
```

```
global f;
a = input(' Valor de a: ');
b = input(' Valor de b: ');
N = input(' No. de simulacoes: ');
k = 0;
```

```
for j = 1:N,  
    x = a*rand; y = b*rand;  
    fun2(a,b,x);  
    if y <= f  
        k = k + 1;  
    end;  
end;  
area = 4.*a*b*k/N      % o valor exato e pi*a*b
```

13 EXERCÍCIOS PROPOSTOS

01) Dada a matriz 3×3 , simétrica e definida positiva, $A = \begin{bmatrix} 6 & -1 & 0 \\ -1 & 4 & 1 \\ 0 & 1 & 6 \end{bmatrix}$, e dado o vetor

de termos independentes, $b = \begin{bmatrix} -1 \\ 1 \\ 2 \end{bmatrix}$,

- use o comando `chol(A)` e obtenha a fatoração Cholesky da matriz A ;
- verifique que $A = C^T C$ usando o comando `transpose`;
- a partir dos fatores Cholesky de A , calcule a solução $x = [x_1 \ x_2 \ x_3]^T$ de $Ax = b$.

02) Dada a matriz literal 3×3 , $A = \begin{bmatrix} a+d & a & 0 \\ a & a-d & -a \\ 0 & -a & d \end{bmatrix}$, e dado o vetor de termos

independentes, também literal, $b = \begin{bmatrix} a+d \\ a \\ 0 \end{bmatrix}$, utilize os recursos da matemática simbólica

para obter a solução $x = [x_1 \ x_2 \ x_3]^T$.

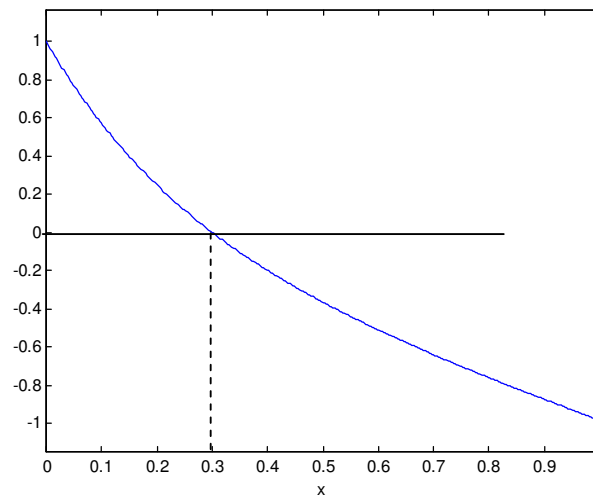
Primeiro aplique a função `inv(A)` para obter x . Em seguida, empregue as funções `simplify` e `pretty` para melhorar a visualização da resposta.

03) Dados $A = \begin{bmatrix} 1 & 3 & 2 \\ 2 & 1 & -1 \\ -1 & 2 & 3 \end{bmatrix}$ e $b = \begin{bmatrix} 7 \\ 5 \\ 4 \end{bmatrix}$, utilize o comando `rref` sobre a matriz

aumentada para obter forma escalonada reduzida e mostre que o sistema $Ax = b$ é incompatível. Para maior clareza, sugere-se que, antes de empregar o comando `rref`, declare os cálculos na forma de frações usando `format rat`.

04) Determine o zero da função transcendental $f(x) = e^{-4x} - x$ utilizando a função `fzero` com estimativa inicial 0.1 . Confira no gráfico através de

```
>> ezplot('exp(-4*x)-x',0,1).
```



05) Dada a equação diferencial $\frac{d^2x(t)}{dt^2} + 3\frac{dx(t)}{dt} + 2x(t) = 0$, com $x(0) = 1$ e $\frac{dx(0)}{dt} = 0$.

Utilize o comando **dsolve** para obter a solução $x(t)$. Em seguida, visualize o gráfico da resposta obtida, $x(t)$, no intervalo $0 \leq t \leq 2$.

06) Dados os binômios $x+1$, $x-2$ e $x+4$. Multiplique-os usando **conv**. A partir do polinômio que resultou da multiplicação calcule as raízes usando **roots**.

07) Desejamos calcular a solução analítica do seguinte sistema de equações

$$\begin{cases} x^2 + y^2 = 1 \\ 1 + 2xy = (a+b)^2 \\ a^2 + b^2 = 1 \end{cases}$$

Para tal, utilize o comando **solve**:

```
>> [b,x,y]=solve('x^2+y^2=1','1 +  
2*x*y=(a + b)^2','a^2 + b^2=1').
```

As soluções esperadas são: $x = a$, $y = b$ ou $x = -a$, $y = -b$.

08) Sabemos que a derivada primeira da função $f(x) = \operatorname{tg}(x)$ é $\frac{df}{dx} = 1 + \sec^2(x)$, sendo $\sec(x) = 1/\cos(x)$. Verifique este fato usando o comando `diff('sin(x)/cos(x)')` seguido logo após pelo comando `simplify(ans)`.

09) Calcule a área sobe a curva da função $f(x) = \frac{4}{1+x^2}$ no intervalo $0 \leq x \leq 1$. Use o comando `int` com limites de integração definidos.

10) No Curso de Cálculo Diferencial e Integral, o professor ensina que a seguinte integral indefinida

$$\int \frac{b}{a^2 + x^2} dx = \frac{b}{a} \operatorname{arc} \operatorname{tg}(x/a)$$

pode ser solucionada aplicando substituição trigonométrica. Comprove o resultado mostrado anteriormente utilizando matemática simbólica e o comando `int` sem limites de integração.

11) Utilize o comando `pretty(diff(1/a*atan(x/a)*b,x))` e verifique que o resultado do exercício 10 está correto.

12) Um importante conceito associado com a derivada de funções escalares n -dimensionais é o gradiente. Digite a seqüência de instruções no *prompt* do MATLAB:

```
[x,y] = meshgrid(-2:.2:2, -2:.2:2);
z = x .* exp(-x.^2 - y.^2);
[px,py] = gradient(z,.2,.2);
```

Para entender melhor o que foi calculado com as instruções anteriores utilize `surf(x,y,z)`.

14 BIBLIOGRAFIA

- [1] HANSELMAN, Duane e LITTLEFIELD, Bruce. "MATLAB 5 – Versão do Estudante, Guia do Usuário", Makron Books, São Paulo, 1999. 413 p. ISBN: 85-346-1058-4.
- [2] ALVES, Antônio César Baleeiro. Notas de Aulas do Curso de Circuitos Elétricos da EEEC/UFG.
- [3] RESNICK, Robert e HALLIDAY, David, "Física 1," 3ª ed., Livros Técnicos e Científicos Editora S.A., Rio de Janeiro, 1981, ISBN: 85-216-0076-3.
- [4] CHAPMAN, S. J. "Programação em MATLAB para engenheiros". Thomson Pioneira.
- [5] MATSUMOTO, E. Y. "MATLAB 7", Editora Érica.
- [6] UNIVERSIDADE DO ESTADO DO RIO DE JANEIRO (UERJ). "Curso de MATLAB 5.1: Introdução à solução de problemas de engenharia". 2ª edição. 181p.
- [7] VALLE, Walkíria Nascente. Notas de Aulas do Curso de Laboratório de Controle de Processos do CMP/UCG.